

PPS - Scala Stream Fusion

Leonardo Naddei

5 ottobre 2026

Indice

1	Introduzione	3
1.1	Obiettivi e criteri di valutazione	4
2	Analisi dei requisiti	5
2.1	Requisiti di business	5
2.2	Requisiti funzionali utente	5
2.3	Requisiti funzionali di sistema	6
2.4	Requisiti negativi ed esclusioni (Out of Scope)	6
2.5	Requisiti non funzionali	7
3	Background e tecnologie abilitanti	8
3.1	Stream fusion	8
3.1.1	Push stream	8
3.1.2	Pull stream	9
3.2	Metaprogrammazione in Scala 3	9
3.2.1	Introduzione alla Multi-stage programming in Scala 3	9
3.3	Metaprogrammazione analitica	10
3.3.1	La reflection API e l'AST tipizzato	11
3.3.2	Tipi staged e tipi riflessivi	11
3.3.3	Beta reduction delle funzioni	12
3.4	Definizione di variabili	12
4	Design architetturale	13
4.1	Architettura generale	13
4.2	Componenti del sistema	13
4.3	Architettura del compilatore	14
5	Design di dettaglio	15
5.1	Design dell'API pubblica	15
5.1.1	Operazioni terminali	15
5.1.2	Tipologie di stream	16
5.1.3	Vincoli negativi sui tipi	16
5.1.4	Operazione terminale <code>collect</code>	17
5.1.5	Configurazione compile-time e runtime	18
5.1.6	Import semplificato dell'API	18
5.2	Design della Stream IR	19
5.2.1	Struttura dell'albero	19
5.2.2	Ast e AstExt	20
5.3	Rappresentazione per la Code Generation (OP Ir)	21
5.3.1	Valori	21
5.3.2	Operazioni	22
5.4	La collection strategy	22

6	Implementazione	24
6.1	Parsing	24
6.1.1	Parsing della stream	24
6.1.2	Parsing del collector	25
6.1.3	Parsing della configurazione statica	26
6.2	Enrichment	26
6.2.1	Materializzazione delle espressioni	27
6.2.2	Stato e condizioni di terminazione	27
6.2.3	Analisi della cardinalità	27
6.3	Traduzione in OP IR	28
6.3.1	Selezione della strategia terminale	28
6.3.2	Generazione della pipeline e delle sorgenti	29
6.3.3	Generazione del piano parallelo	30
6.4	Lowering	30
6.4.1	Traduzione dell'OP IR	30
6.4.2	Applicazione e inlining delle funzioni	31
6.4.3	Chiamate e inlining dei collector	31
6.4.4	Lowering del parallelismo	31
6.5	Esempio di traduzione end-to-end	32
6.5.1	L'input alla libreria	32
6.5.2	Il risultato del parsing	34
6.5.3	L'output dell'enrichment	34
6.5.4	Generazione dell'OpIR	35
6.5.5	Lowering verso l'AST Scala	36
6.5.6	Il bytecode JVM generato	37
7	Testing e valutazione delle prestazioni	39
7.1	Testing	39
7.2	Valutazione delle prestazioni	40
7.2.1	Metodologia	40
7.2.2	Benchmark sequenziali	40
7.2.3	Benchmark paralleli	42
7.2.4	Comportamento a freddo e livelli di compilazione JIT	43
7.2.5	Allocazioni e comportamento del Garbage Collector	44
8	Retrospettiva	46
8.1	Limiti e compromessi progettuali	46
8.2	Sviluppi futuri	47
8.3	Conclusioni	47

Capitolo 1

Introduzione

Il modello di programmazione basato su pipeline dichiarative è sempre più diffuso per l'elaborazione di collezioni e sequenze di dati. Operazioni come `map`, `filter` e `flatMap` permettono di descrivere una computazione separando la logica dell'elaborazione dai dettagli relativi all'attraversamento della struttura dati.

Questa forma di astrazione migliora la leggibilità e la composizionalità del codice, ma può introdurre un costo rispetto alla tipica implementazione imperativa. In particolare, sulla JVM, una pipeline ad alto livello può richiedere la costruzione di strutture intermedie, l'allocazione di closure, l'invocazione indiretta delle funzioni associate agli operatori e, in alcuni casi, operazioni di boxing e unboxing. Il compilatore JIT può eliminare parte di questi costi, ma la sua efficacia dipende dalla struttura del programma.

L'impatto è osservabile anche in sistemi reali. Nel progetto Apache Cassandra sono state, ad esempio, introdotte ottimizzazioni mirate alla rimozione delle Java Stream da percorsi di esecuzione particolarmente frequenti [4] e all'eliminazione di capturing lambda [3]. Questi interventi mostrano come possa risultare necessario rinunciare ad astrazioni di alto livello per ottenere un'esecuzione più efficiente.

Il limite dei sistemi basati sull'applicazione dinamica di lambda function emerge, in particolare, quando lo stesso call site viene utilizzato con più più lambda differenti. In questi casi il call site diventa **polimorfico** o **megamorfico**. Il problema è noto e sono state proposte diverse strategie per preservare e propagare maggiori informazioni di specializzazione. Tra queste, la proposta della *Parametric VM* discute meccanismi per associare informazioni di specializzazione a metodi e call site, consentendo alla JVM di generare codice maggiormente specializzato [7]. La libreria evita invece di affrontare questo problema a runtime: quando la funzione è nota staticamente, ne effettua l'inlining e la monomorfizzazione a compile-time, eliminandone il dispatch dal codice generato.

Il problema affrontato in questo progetto consiste quindi nel ridurre il divario tra due modalità di esprimere la stessa computazione: da un lato una pipeline dichiarativa e composizionale, dall'altro un ciclo imperativo esplicitamente specializzato per l'elaborazione da eseguire. L'obiettivo è quindi preservare il primo modello di programmazione producendo codice vicino al secondo.

Il principio alla base della trasformazione è quello della *stream fusion*, introdotto da Coutts et al. in *Stream Fusion: From Lists to Streams to Nothing at All* [1]: le operazioni intermedie vengono fuse in un singolo flusso di esecuzione, evitando, la materializzazione dei risultati intermedi.

Per certi aspetti, il modello ottenuto richiama la monomorfizzazione a compile-time dei generics adottata da Rust e i template in C++: il codice viene specializzato rispetto ai tipi concretamente utilizzati. FusedStream estende questo principio alla struttura stessa dell'esecuzione: analizzando la composizione della pipeline e generando un'implementazione specializzata rispetto alla sorgente, agli operatori e all'operazione terminale.

Particolare attenzione è stata dedicata alla progettazione dell'API. Le proprietà e i vincoli della pipeline vengono rappresentati attraverso il type system. La libreria è inoltre estendibile da parte degli utilizzatori. In particolare le operazioni terminali disponibili nativamente possono essere estese tramite interfacce dedicate.

1.1 Obiettivi e criteri di valutazione

L'obiettivo del progetto è verificare se sia possibile preservare l'espressività di una pipeline dichiarativa generando codice con caratteristiche prossime a quelle di un'implementazione imperativa. La valutazione considera quindi tre aspetti principali: la correttezza semantica della trasformazione, le caratteristiche del codice generato e le prestazioni a runtime.

La correttezza viene verificata mediante test funzionali e di compilazione. L'ispezione del codice e del bytecode generato permette invece di verificare l'effettiva fusione delle operazioni, la specializzazione delle chiamate e l'eliminazione di allocazioni e dispatch non necessari. Infine, benchmark dedicati confrontano il throughput con implementazioni imperative equivalenti e con le corrispondenti pipeline offerte dalle librerie standard.

Capitolo 2

Analisi dei requisiti

2.1 Requisiti di business

- RB-1** **Colmare il divario tra espressività e prestazioni.** La programmazione dichiarativa tramite pipeline favorisce la leggibilità, l'astrazione e la manutenibilità. Tuttavia, sulla JVM tali astrazioni comportano comunemente una penalizzazione prestazionale. L'obiettivo primario del progetto è annullare questo compromesso, consentendo di scrivere codice ad alto livello dichiarativo che compili direttamente in codice a basso.
- RB-2** **Adozione a basso attrito.** L'integrazione della libreria non deve richiedere modifiche al toolchain standard del compilatore né l'apprendimento di API complesse. La sintassi offerta deve risultare familiare a chi già utilizza le collezioni standard di Scala o le API di Java Stream, garantendo un'interfaccia compatta, uniforme e accessibile tramite un singolo import.

2.2 Requisiti funzionali utente

- RFU-1** **Creazione dello Stream.** L'utente deve poter effettuare elaborazioni a partire da diverse sorgenti di dati (ad esempio collezioni Scala, array o iteratori) attraverso un unico punto di ingresso.
- RFU-2** **Configurazione.** L'utente deve poter configurare il comportamento del sistema, sia per quanto riguarda i parametri di generazione statica del codice che per i parametri riguardanti l'esecuzione a runtime. In assenza di configurazione esplicita, il sistema deve utilizzare valori predefiniti senza richiedere ulteriori interventi da parte dell'utente.
- RFU-3** **Esecuzione parallela esplicita.** Qualora la sorgente dei dati sia adatta, l'utente deve poter richiedere l'esecuzione parallela della pipeline.
- RFU-4** **Raccolta dei risultati.** L'utente deve poter determinare il risultato tramite funzioni predefinite della libreria (come ad esempio collezionando in lista) ma anche in maniera personalizzata tramite interfacce specifiche.
- RFU-5** **Guida contestuale tramite Type System.** L'API deve guidare attivamente l'utente durante la costruzione della pipeline sfruttando il type system. Tramite tipi e subtyping avanzato, l'IDE e il compilatore devono esporre ed accettare solo ed esclusivamente le operazioni valide per lo stato corrente dello stream (es. inibendo metodi non supportati in parallelo).

- RFU-6** **Comportamento prevedibile dell'API.** Le operazioni esposte dalla libreria devono seguire semantiche coerenti con quelle comunemente associate alle pipeline di elaborazione di collezioni, minimizzando comportamenti inattesi per l'utente (*Least Astonishment*). Operazioni equivalenti a quelle offerte dalle API standard devono, ove applicabile, preservarne il significato e le aspettative d'uso.

2.3 Requisiti funzionali di sistema

- RFS-1** **Riconoscimento e analisi della pipeline statica.** Il sistema deve essere in grado di ispezionare a compile time l'albero sintattico di una pipeline dichiarativa e scomporla nella sequenza logica di sorgente, trasformazioni intermedie e operazione terminale.
- RFS-2** **Fusione delle operazioni sequenziali.** Il sistema deve collassare le trasformazioni in un unico flusso di esecuzione continuo, eliminando la materializzazione di collezioni o iteratori tra uno stadio e il successivo.
- RFS-3** **Generazione di codice specializzato.** Il sistema deve emettere codice specializzato per i tipi di dato coinvolti e per l'operazione terminale richiesta, evitando chiamate polimorfiche o indirette.
- RFS-4** **Gestione del flusso e terminazione anticipata.** Il sistema deve regolare il flusso di controllo del ciclo generato per supportare sia l'elaborazione esaustiva dell'intero flusso sia l'interruzione anticipata quando la logica dell'operazione terminale o di filtro lo richiede.
- RFS-5** **Composizione di flussi annidati.** Il sistema deve consentire la fusione di flussi generati dinamicamente da elementi intermedi (`flatMap`), integrando il ciclo interno nel contesto del ciclo esterno senza creare buffer di accumulazione intermedi o computazioni superflue.
- RFS-6** **Supporto all'esecuzione parallela.** Per le sorgenti e i terminali compatibili, il sistema deve emettere codice che esegua la computazione in maniera parallela.

2.4 Requisiti negativi ed esclusioni (Out of Scope)

Al fine di garantire la predicibilità prestazionale e la robustezza del compilatore, sono definiti i seguenti punti come out of scope:

- RN-1** **Assenza di composizione dinamica runtime.** Il sistema non deve supportare la costruzione dinamica di pipeline determinate da rami condizionali ordinari a runtime (ad esempio `if (cond) s.map(...) else s.filter(...)`). L'intera catena di trasformazioni deve essere fissa e nota sintatticamente al call-site di compilazione.
- RN-2** **Nessuna ottimizzazione opaca sul codice utente.** Il sistema non deve alterare né riscrivere la logica interna delle espressioni arbitrarie fornite dall'utente all'interno delle lambda.

- RN-3** **Nessun supporto a flussi annidati di stream.** Il sistema non supporta la generazione di stream i cui singoli elementi siano a loro volta istanze di stream tramite trasformazioni mapping (es. `map(x => FusedStream.of(x))`). La sola composizione nested consentita è quella finalizzata all'appiattimento esplicito tramite `flatMap`.
- RN-4** **Errori parlanti e assenza di fall-back a runtime.** Se una pipeline viola i vincoli del framework il compilatore deve rigettare esplicitamente il programma tramite messaggi d'errore posizionati e descrittivi o conversioni nascoste a runtime.
- RN-5** **Nessun inlining o ottimizzazione per funzioni non-lambda.** Qualora a una trasformazione (es. `map`) venga passato una funzione preesistente anziché un'espressione lambda, il compilatore non applica la β -reduction né l'inlining statico.

2.5 Requisiti non funzionali

- RNF-1** **Prestazioni a runtime.** Il codice generato deve esibire un throughput comparabile a codice efficiente scritto manualmente in Java, riducendo l'overhead computazionale rispetto alle pipeline funzionali convenzionali. Per sorgenti a oggetti generici il boxing il comportamento standard della JVM.
- RNF-2** **Assenza di allocazioni intermedie.** L'elaborazione delle operazioni intermedie stateless non deve allocare oggetti sul runtime heap.
- RNF-3** **Trasparenza semantica.** A parità di sorgente, funzioni utente e configurazione, il codice generato deve produrre risultati semanticamente equivalenti a quelli definiti dalla pipeline dichiarativa.

Capitolo 3

Background e tecnologie abilitanti

Questo capitolo introduce i principali concetti teorici e gli strumenti di Scala 3 necessari alla comprensione delle scelte progettuali e implementative adottate nel progetto.

3.1 Stream fusion

Una stream pipeline può essere rappresentata secondo due modelli di esecuzione fondamentali: un modello *push*, guidato dal produttore, e un modello *pull*, guidato dal consumatore. Questa distinzione viene discussa, nel contesto della stream fusion, da Kiselyov et al. in *Stream Fusion, to Completeness* [2].

3.1.1 Push stream

Nel modello *push* è la sorgente a guidare l'elaborazione: ogni elemento prodotto viene immediatamente propagato attraverso gli operatori successivi della pipeline fino a raggiungere il terminale. Concettualmente, l'intera sequenza di operatori può quindi essere vista come una “mega-funzione” applicata a ciascun elemento della sorgente.

Per esempio:

```
1 stream
2   .filter(p)
3   .map(f)
4   .collect(collector)
```

può essere interpretata, come una singola funzione data dalla composizione:

$$\text{consume}(x) = \text{collector}(\text{map}(\text{filter}(x))).$$

Questa rappresentazione rende particolarmente semplice l'implementazione di operatori come **map** e **filter**: ciascun operatore può scartare oppure inoltrare (eventualmente trasformandolo) l'elemento ricevuto.

E' più complessa la gestione di operatori che modificano la cardinalità della stream: operazioni come **skip** e **limit** richiedono infatti di mantenere uno stato relativo al numero di elementi osservati. Nel caso di **limit**, una volta raggiunto il numero di elementi, è necessario interrompere anticipatamente la produzione della sorgente.

Una difficoltà ancora maggiore emerge con operatori che combinano più stream come lo **zip**. Nel modello push ciascuna sorgente possiede infatti un proprio flusso di produzione: per ogni elemento che viene emesso dalla sorgente si può avere un numero diverso di elementi prodotti nel punto di zipping. Per associare l'elemento corrente di una stream con quello corrispondente di un'altra è necessario quindi introdurre un meccanismo di sincronizzazione tra i due produttori.

3.1.2 Pull stream

Nel modello *pull*, al contrario, il controllo dell'esecuzione è affidato al consumatore. La stream viene rappresentata come un oggetto capace di produrre, su richiesta, l'elemento successivo. In questo caso non è la sorgente a inviare gli elementi verso gli operatori successivi, ma è il terminale a richiederli risalendo progressivamente la pipeline. Ogni operatore intermedio definisce quindi il proprio comportamento in termini dell'operatore precedente: una `map`, ad esempio, richiede un elemento alla stream sottostante e applica ad esso la funzione di trasformazione. Un `filter` continua invece a richiedere elementi finché non ne incontra uno che soddisfa il predicato.

Questa organizzazione rende naturali operazioni come `limit`, poiché il consumatore può semplicemente cessare di richiedere nuovi elementi, e soprattutto operatori come `zip`: è sufficiente richiedere contemporaneamente un elemento a ciascuna delle due stream e combinarne i risultati. Il prezzo da pagare è una rappresentazione più esplicita dello stato dell'iterazione e, in assenza di successive ottimizzazioni, un maggior numero di passaggi intermedi necessari alla produzione di ciascun elemento.

3.2 Metaprogrammazione in Scala 3

Il progetto richiede sia la generazione di nuovo codice sia l'analisi e la trasformazione del codice ricevuto dalla macro. Questa sezione introduce i meccanismi di metaprogrammazione di Scala 3 utilizzati a tale scopo. Quanto segue e gli esempi riportati fanno riferimento al modello di metaprogrammazione generativa e analitica presentati in [8, 9]

3.2.1 Introduzione alla Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*.

Una quote `'{ expression }` costruisce una rappresentazione del codice contenuto al suo interno. Se `expression` produce un valore di tipo `T`, il risultato della quote ha tipo `Expr[T]` e rappresenta codice che produrrà un valore di tipo `T`.

Uno splice: ``${expression}`` opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione `Expr[T]`.

Un esempio di generazione staged è l'elevamento a potenza con esponente noto a compile time:

```
1 def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =  
2   if n == 0 then  
3     Expr(1)  
4   else  
5     '{ $x * ${ powCode(x, n - 1) } }
```

L'espressione

```
'{ $x * ${ powCode(x, n - 1) } }
```

combina codice generato e computazioni eseguite durante l'espansione della macro. In particolare:

- `'{ ... }'` è la *quote* esterna: costruisce un nuovo valore `Expr[Int]` che rappresenta l'espressione di moltiplicazione generata.
- `x` è un parametro di tipo `Expr[Int]` e rappresenta quindi un frammento di codice, non direttamente un valore intero. Lo splice `$x` inserisce tale frammento come operando della moltiplicazione.

- `powCode(x, n - 1)` è invece una computazione eseguita ricorivamente durante l'espansione della macro.
- lo splice `${ powCode(x, n - 1) }` inserisce il codice prodotto dalla chiamata ricorsiva come operando destro moltiplicazione;
- l'operatore `*` appartiene infine al codice contenuto nella quote: non viene eseguito durante la generazione, ma diventa parte del programma generato.

La chiamata a `powCode` e la sua ricorsione avvengono a *compile time*, le moltiplicazioni costruite all'interno delle quote saranno eseguite a *runtime*.

Ad esempio, per `n = 3`, le successive espansioni possono essere viste come:

```

1 powCode(x, 3)
2 => '{ $x * ${powCode(x, 2)} }'
3 => '{ $x * $x * ${powCode(x, 1)} }'
4 => '{ $x * $x * $x * ${powCode(x, 0)} }'
5 => '{ $x * $x * $x * 1 }'

```

3.3 Metaprogrammazione analitica

Una macro generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione, come nell'esempio di `powCode` appena visto. È tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma e analizzarne la struttura.

Scala permette di farlo in maniera elementare attraverso il pattern matching su codice quoted. Una espressione può essere analizzata come segue:

```

1 expr match {
2   case '{ someFunction($argument) } =>
3   ...
4 }

```

Il pattern quoted descrive la struttura sintattica che si intende riconoscere. Le parti precedute da `$` costituiscono invece i pattern di frammenti presenti nelle corrispondenti posizioni dell'espressione da catturare.

Ad esempio, un'addizione tra due espressioni intere:

```

1 def inspect(expr: Expr[Int])(using Quotes): Unit = expr match
2   case '{ ($left: Int) + ($right: Int) } => ...
3   case _ => println("Not recognized")

```

Applicando `inspect` all'espressione `'{ x + 42 }` il pattern riconosce l'operazione di addizione e associa a `left` e `right` i due frammenti:

$$\text{left} \longrightarrow \text{'}\{x\} \quad \text{right} \longrightarrow \text{'}\{42\}.$$

Entrambe le variabili hanno tipo `Expr[Int]`: non contengono i valori ma rappresentano il codice corrispondente.

Quote e splice assumono quindi un ruolo complementare anche nell'analisi: all'interno dei quoted patterns, la quote descrive la forma del codice da riconoscere, mentre gli splice identificano le porzioni variabili della struttura e le rendono disponibili come valori `Expr[T]` per le successive fasi della macro.

Scala offre anche una modalità alternativa e più generale per l'analisi del codice: la *Reflection API*, che consente di accedere direttamente alla rappresentazione dell'albero sintattico prodotto dal compilatore.

3.3.1 La reflection API e l'AST tipizzato

Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST del compilatore con `expr.asTerm`. La Reflection API espone la struttura prodotta dal compilatore, i nodi più importanti utilizzati nel parser sono:

- `Apply`, per l'applicazione di una funzione;
- `TypeApply`, per un'applicazione contenente parametri di tipo;
- `Select`, per la selezione di un membro;
- `Ident`, per il riferimento a un identificatore;
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale;
- `Inlined`, per rappresentare il risultato di un'espansione inline;
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice.

Ad esempio, l'espressione `numbers.map(f)` può essere rappresentata, in forma semplificata, dal seguente albero:

```
1 Apply(  
2   Select(Ident("numbers"), "map"),  
3   List(Ident("f"))  
4 )
```

Tale struttura può essere analizzata direttamente mediante pattern matching:

```
1 expr.asTerm match  
2   case Apply(Select(source, "map"), List(function)) => ...
```

Il pattern permette quindi di riconoscere l'operazione `map` e di estrarre separatamente il termine `source`, che rappresenta la sorgente della chiamata, e `function`, che ne rappresenta l'argomento. L'analisi programmatica dell'AST consente quindi di riconoscere e manipolare strutture più complesse e variabili rispetto al quoted pattern matching, senza dover descrivere a priori l'intera forma sintattica dell'espressione mediante un singolo pattern.

3.3.2 Tipi staged e tipi riflessivi

Scala mette a disposizione due rappresentazioni dei tipi:

Type[T] rappresenta il tipo `T` nell'API staged ed è utilizzabile insieme a `Expr[T]`, quote e splice;

TypeRepr rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Il passaggio dalla rappresentazione riflessiva a quella staged avviene mediante `asType`. Data una `TypeRepr`, questo metodo restituisce un `Type[?]`, sul quale è possibile effettuare pattern matching per rendere disponibile il tipo sottostante:

```

1 item.asType match {
2   case '[t] => ...
3 }

```

Il pattern `'[t]` introduce `t` come tipo utilizzabile all'interno del relativo ramo. Questo permette, ad esempio, di partire da un termine dell'AST di tipo non noto a priori e generare nuovo codice che ne preservi il tipo:

```

1 def makeList(term: Term)(using Quotes): Expr[Any] = term.tpe.asType match
2   case '[t] =>
3     val value: Expr[t] = term.asExprOf[t]
4     '{ List[t]($value) }

```

Dato il tipo di `term`, `term.tpe` ne fornisce la rappresentazione riflessiva `TypeRepr`. Il pattern `'[t]` rende tale tipo disponibile come `t`. A questo punto `t` può essere utilizzato all'interno di una quote per costruire nuovo codice tipizzato.

Questo meccanismo fa da collegamento tra Reflection API e staged programming: un tipo individuato durante l'analisi dell'AST può essere portato nel contesto staged e utilizzato per costruire nuove espressioni preservandone l'informazione di tipo.

3.3.3 Beta reduction delle funzioni

Un altro elemento importante è la funzionalità di β -reduction. Data una funzione:

$$f = x \mapsto e, \quad (3.1)$$

$f(a)$ può calcolata per sostituzione, sostituendo direttamente x con a all'interno di e . Scala rende disponibile questa trasformazione attraverso: `Expr.betaReduce(...)`. Incorpora staticamente il corpo della lambda nell'IR generato, la macro eliminando sia la rappresentazione della funzione come entità invocabile sia il dispatch dell'invocazione.

3.4 Definizione di variabili

Nell'AST di Scala, un riferimento a una variabile è associato a un `Symbol`. Il nome è solamente una proprietà del simbolo e non ne rappresenta l'identità. La Reflection API permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`. La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto nè con variabili omonime definite nel codice utente nè con altre variabili create durante l'espansione della macro.

Capitolo 4

Design architetturale

4.1 Architettura generale

Il progetto è organizzato in due aree principali: l'*API pubblica* e il *compilatore*.

L'*API pubblica*, descritta nella Sezione 5.1, espone il modello con cui l'utente definisce la pipeline. L'invocazione di un'operazione terminale attiva il compilatore, che analizza staticamente la pipeline e genera il codice necessario alla sua esecuzione.

4.2 Componenti del sistema

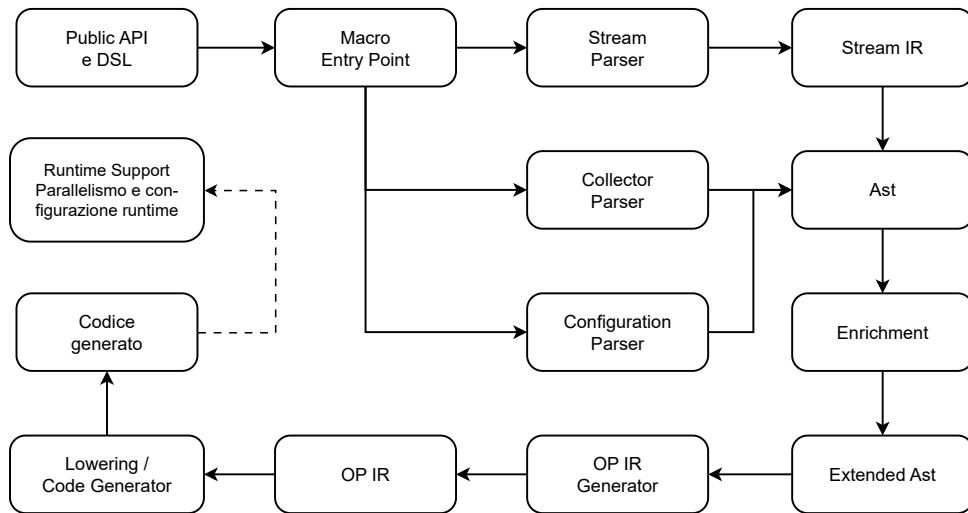


Figura 4.1: Architettura generale e principali componenti del sistema. Le frecce continue rappresentano il flusso di compilazione della pipeline, le frecce tratteggiate indicano le dipendenze a runtime.

Come mostrato in Figura 4.1, il flusso di compilazione parte dalla Public API e termina con la produzione del codice eseguibile. Il *Macro Entry Point* costituisce il punto di ingresso del compilatore e orchestra le diverse fasi di trasformazione.

Il *Parser* analizza l'AST Scala della pipeline e ne ricostruisce la rappresentazione logica nella *Stream IR*. Parallelamente, il collector e la configurazione vengono analizzati per determinare rispettivamente la strategia terminale e i parametri che influenzano la generazione e l'esecuzione del codice. L'output di questa fase è un *Ast* che contiene tutte le informazioni estratte.

L'AST viene successivamente analizzata ed estesa dalla fase di *Enrichment*, che associa alla pipeline le informazioni statiche necessarie alla pianificazione dell'esecuzione, tra cui proprietà

della sorgente, cardinalità e condizioni di terminazione. Questa fase produce un *AstExt* che viene poi passato all'*OP IR Generator* che utilizza questa rappresentazione arricchita per produrre la *Operation IR* (OP IR), nella quale le operazioni logiche della pipeline vengono trasformate in un piano orientato all'esecuzione. Infine, il *Lowering* traduce la OP IR in un nuovo AST Scala, che viene restituito al compilatore e reinserito nel normale processo di compilazione. Il *Runtime Support*, esterno al flusso principale di trasformazione, fornisce invece le funzionalità richieste dal codice generato, in particolare per l'esecuzione parallela e la configurazione runtime.

4.3 Architettura del compilatore

La pipeline interna del compilatore può essere suddivisa concettualmente in *frontend* e *backend*, come mostrato in Figura 4.2.

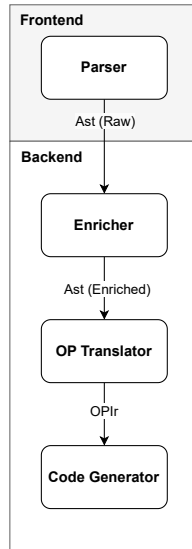


Figura 4.2: Architettura generale del compilatore di FusedStream.

Il *frontend* è responsabile dell'interpretazione della DSL: analizza l'AST Scala ricevuto dalla macro e ricostruisce la struttura della pipeline nella Stream IR. Questa rappresentazione mantiene una corrispondenza diretta con i concetti del dominio, quali sorgenti e operatori intermedi.

Il *backend* trasforma progressivamente questa rappresentazione fino a ottenere il codice eseguibile. L'Enrichment completa la Stream IR con le informazioni statiche necessarie alla generazione; l'OP IR Generator traduce quindi la pipeline arricchita in una rappresentazione orientata all'esecuzione; infine, il Lowering converte la OP IR nel nuovo AST Scala.

Il flusso complessivo del compilatore può essere visto come un flusso di rappresentazioni:

Scala AST $\longrightarrow$ Ast $\longrightarrow$ AstExt $\longrightarrow$ OP IR $\longrightarrow$ Generated Scala AST.

Capitolo 5

Design di dettaglio

5.1 Design dell'API pubblica

L'obiettivo nel design dell'API pubblica è stato quello di fornire un'interfaccia che garantisca contemporaneamente:

- la possibilità di descrivere pipeline sufficientemente espressive
- l'estendibilità, in particolare per quanto riguarda le operazioni terminali
- il maggior numero possibile di garanzie a compile time
- l'eliminazione di codice che verificasse esplicitamente la correttezza delle combinazioni di operazioni richieste dall'utente.

Dal punto di vista concettuale, ogni stream può essere scomposto in tre componenti principali:

1. una sorgente, generalmente costituita da una collezione o da un array;
2. una sequenza di operazioni intermedie (`map`, `filter`, `flatMap`,...)
3. un'operazione terminale

L'organizzazione dei tipi esposti dall'API riflette direttamente questa struttura. Tramite `sealed trait`, tipi parametrici, tipi opachi e vincoli sui parametri di tipo è stato possibile modellare tutte le proprietà della pipeline. In questo modo il compilatore Scala e l'IDE possono guidare l'utente durante la costruzione dello stream e impedire le combinazioni non supportate. L'API mantiene, allo stesso tempo, un certo grado di estendibilità consentendo all'utente di definire nuove operazioni terminali attraverso l'interfaccia `Collector`.

5.1.1 Operazioni terminali

Le operazioni terminali sono rappresentate attraverso l'astrazione `Collector`. La rappresentazione completa è riportata in Figura 5.2.

I terminali sono parametrizzati tramite quattro parametri di tipo che rappresentano rispettivamente:

- E: il tipo degli elementi ricevuti dallo stream;
- B: il tipo dello stato intermedio, o buffer, utilizzato durante l'accumulazione;
- R: il tipo del risultato finale prodotto dal collector;

- **S**: la politica di terminazione del collector.

Il metodo `supplier` crea lo stato iniziale dell'accumulator, `accumulator` aggiunge un nuovo elemento, il valore booleano restituito segnala che l'elaborazione può essere interrotta (si pensi a una `findFirst`). Infine, `finisher` trasforma lo stato intermedio nel risultato finale.

I collector possono essere esaustivi ovvero richiedere l'elaborazione dell'intera sorgente, oppure possono terminare anticipatamente senza consumare tutti i dati. Questa proprietà è rappresentata tramite `TerminationPolicy`, i cui sottotipi `Exhaustive` e `ShortCircuiting` distinguono rispettivamente i collector che richiedono l'elaborazione completa della sorgente da quelli che possono terminare anticipatamente (Figura 5.2).

Collector combinabili e parallelismo

Per poter eseguire una riduzione in parallelo è necessario ma non sufficiente che il collector sia `Exhaustive` serve anche una funzione che permetta di combinare gli stati intermedi prodotti indipendentemente dai diversi worker.

Questa proprietà è rappresentata da `Combinable[B]`. Come mostrato in Figura 5.2, `ParallelCollector` combina le due proprietà richieste, modella infatti un `Collector Combinable[B]` con politica `Exhaustive`. Il type system garantisce quindi che una pipeline parallela possa utilizzare soltanto collector adeguati.

Intrinsic vs. first-class collectors

Alcune operazioni terminali ad alta frequenza d'uso, come `toArray` e `summing`, beneficiano di un trattamento specializzato e assumono il ruolo di veri e propri *intrinsic collectors*. Come illustrato in Figura 5.2, tali operazioni sono modellate rispettivamente mediante i tipi opachi `ToArrayCollector` e `SummingCollector`, entrambi sottotipi di `ParallelCollector`.

A differenza dei collector convenzionali, queste entità non operano semplicemente come valori gestiti a runtime tramite l'interfaccia `Collector`: la loro presenza lungo la pipeline viene intercettata staticamente dal compilatore, innescando una strategia di code generation dedicata. L'adozione di tipi opachi consente quindi di preservare la natura semantica dell'*intrinsic* esponendo a tempo di compilazione proprietà chiave come il supporto all'esecuzione parallela pur mantenendo incapsulati i dettagli implementativi.

Al contrario, soluzioni predefinite quali `toList` e `toSet` rimangono puri costrutti di prima classe: non godono di alcuna conoscenza privilegiata da parte del compilatore e sono gestite al pari di qualsiasi collector definito dall'utente tramite l'interfaccia pubblica.

5.1.2 Tipologie di stream

La modalità di esecuzione della pipeline è rappresentata staticamente attraverso la gerarchia di tipi mostrata in Figura 5.1. `SequentialStream` e `ParallelStream` distinguono rispettivamente le pipeline sequenziali e parallele, mentre `ParallelizableSource` identifica le sorgenti sequenziali per le quali è disponibile la transizione alla modalità parallela.

I sottotipi preservano la modalità di esecuzione lungo la pipeline: ad esempio, una `map` applicata a una `ParallelStream[A]` produce una `ParallelStream[B]`. Analogamente, la disponibilità di alcune operazioni è vincolata dal tipo statico: `skip` e `limit`, definite esclusivamente per `SequentialStream`, non sono disponibili su una `ParallelStream`.

5.1.3 Vincoli negativi sui tipi

Alcune proprietà dell'API sono espresse mediante vincoli negativi tramite `NotGiven` sui parametri di tipo. In particolare si richiede che una determinata evidenza implicita *non* sia

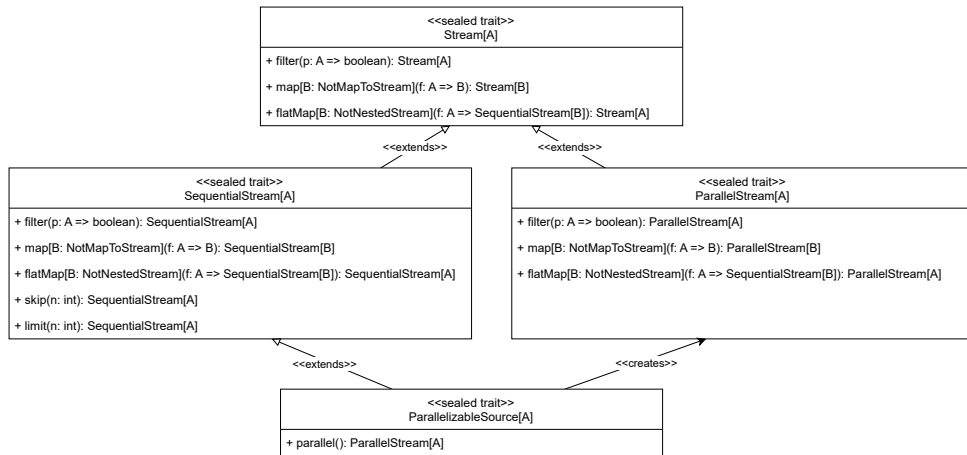


Figura 5.1: Gerarchia dei tipi utilizzati per rappresentare le modalità di esecuzione degli stream.

disponibile. Questo meccanismo viene utilizzato per impedire la costruzione di stream annidati attraverso operazioni. Ad esempio, il metodo `map` è definito come:

```
1 def map[B: NotMapToStream](f: A => B): Stream[B]
```

dove l'evidenza `NotMapToStream[B]` può essere costruita soltanto quando `B` non è un sottotipo di `Stream[?]`:

```
1 given [T](using NotGiven[T <:: Stream[?]]): NotMapToStream[T] = new NotMapToStream[
  T] {}
```

Di conseguenza, un'espressione come `stream.map(x => FusedStream.from(...))` viene rifiutata a compile time.

Tramite le annotazioni `@implicitNotFound` vengono emessi messaggi di errore chiari e specifici. In questo modo combinazioni non supportate vengono escluse direttamente durante la risoluzione dei tipi, senza richiedere controlli espliciti nella successiva fase di trasformazione della pipeline.

5.1.4 Operazione terminale `collect`

L'operazione `collect`, definita tramite extension methods, costituisce il punto cardine nel quale la descrizione dichiarativa della pipeline viene trasformata staticamente nel codice bytecode imperativo che ne realizza l'esecuzione.

Una pipeline sequenziale può utilizzare genericamente un `Collector`, mentre una pipeline parallela richiede un `ParallelCollector`, ovvero un collector esaustivo i cui risultati parziali siano combinabili. Questa distinzione permette al type system di rifiutare staticamente combinazioni non supportate:

```
1 stream.parallel().collect(Collector.findFirst) // rifiutato a compile-time
2 stream.parallel().collect(Collector.toArray)  // valido
```

`findFirst` è infatti marcato come `ShortCircuiting` e non soddisfa i requisiti di `ParallelCollector`, mentre `ToArrayCollector[T]` ne è un sottotipo esaustivo. La gerarchia completa dei collector e delle relative politiche di terminazione è illustrata in Figura 5.2.

Inlining dei Collector e la modalità `strictInlining`. Per ridurre le allocazioni e il dispatch dinamico a runtime, la libreria effettua l'inlining delle chiamate ai metodi del collector

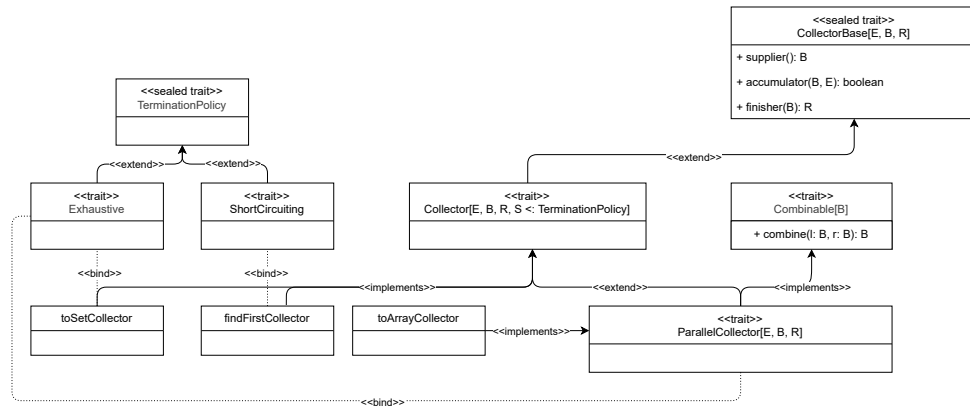


Figura 5.2: Gerarchia dei Collector, politiche di terminazione e vincoli di inlining. La relazione «bind» rappresenta la specializzazione statica di un parametro di tipo (ad es. *S* con *Exhaustive* o *ShortCircuiting*).

per default (modalità `strictInlining = true`). In questo caso il collector fornito a `collect` deve essere una classe concreta rifiutando classi anonime, classi astratte e trait e le funzioni del collector devono essere obbligatoriamente dichiarate come `inline def`.

Queste condizioni sono verificate a compile time, durante l'esecuzione della macro. Questa modalità può essere disabilitata tramite `strictInlining = false`, in questo caso rimarranno le chiamate dinamiche ai metodi del collector.

5.1.5 Configurazione compile-time e runtime

La libreria distingue tra due tipologie di configurazione:

CompileConfig (Configurazione di compilazione). Determina le strategie di manipolazione dell'AST e le ottimizzazioni applicate dalla macro a compile time:

- **useUnsafe:** quando abilitato, sfrutta `VarHandle` per accedere direttamente al vettore interno di `java.util.ArrayList`
- **enableLogging:** abilita il logging dell'esecuzione della macro
- **strictInlining:** Abilita l'inlining delle chiamate ai collector

RuntimeConfig (Configurazione di esecuzione). Determina i parametri dell'esecuzione:

- **executionContext:** il thread pool utilizzato per l'esecuzione parallela
- **workerCount:** il numero di worker paralleli
- **chunksPerWorker:** il fattore moltiplicativo la divisione in chunks

In assenza di configurazioni esplicite, la libreria fornisce istanze given predefinite, garantendo il minimo attrito nell'uso per le pipeline standard

5.1.6 Import semplificato dell'API

Tutti gli elementi dell'API pubblica vengono riesposti direttamente attraverso l'oggetto `FusedStream`. Questo permette di importare l'intera API necessaria con una singola istruzione:

```
1 import FusedStream.*
```

ottenendo così accesso alle funzioni per la costruzione delle pipeline, ai collector terminali pronti all'uso e agli oggetti di configurazione. L'obiettivo è mantenere l'interfaccia compatta e immediata, evitando che l'utente debba importare esplicitamente i diversi oggetti che compongono l'API.

5.2 Design della Stream IR

L'obiettivo della *Stream Intermediate Representation* (Stream IR) è rappresentare la struttura logica di una pipeline e, permettere alle successive fasi del compilatore di associare all'albero nuove informazioni.

Il problema principale da gestire è la mutazione di Abstract Syntax Tree durante l'intero processo di compilazione. Le fasi del compilatore introducono informazioni da associare ai nodi dell'albero. Una possibile soluzione consiste nel definire una rappresentazione differente per ciascuna fase. Questo porterebbe però duplicazione di una parte considerevole della struttura dell'AST. Un'altra opzione potrebbe essere avere un nodo unico per ogni fase con campi opzionali o un nodo base con varie estensioni per reappresentare le proprie fasi, questi approcci hanno però un problema comune: rendono più complicata ed error prone la gestione di queste informazioni.

Il design utilizzato per la Stream IR è invece una struttura ad albero parametrizzata che prende ispirazione dal paper *Trees That Grow* [5].

```
1 sealed trait Phase
2 object Phase {
3   sealed trait Raw extends Phase
4   sealed trait Enriched extends Phase
5 }
6 enum StreamTree[P <: Phase, A] {
7   ...
8 }
```

A Rappresenta il tipo degli elementi prodotti dal nodo.

P Identifica la fase del ciclo di vita (**Phase**) nella quale il nodo si trova.

L'utilizzo della fase come parametro di tipo rende esplicito nel type system lo stato della rappresentazione e, di conseguenza, quali informazioni siano disponibili e quali operazioni siano valide in ciascuna fase. Questo permette di ottenere garanzie a compile time sulla corretta manipolazione della Stream IR e di sfruttare il pattern matching esaustivo sulle diverse rappresentazioni, evitando al contempo il ricorso a campi opzionali o gerarchie di nodi distinte per ciascuna fase.

5.2.1 Struttura dell'albero

La Stream IR rappresenta una pipeline come una struttura ad albero, i cui nodi modellano le operazioni e le sorgenti dello stream. La struttura è prevalentemente lineare, ma può ramificarsi quando un'operazione introduce uno stream annidato, rappresentato mediante un sottoalbero aggiuntivo. Una rappresentazione semplificata della gerarchia è mostrata in Figura 5.3.

Gli operatori che non modificano la propria rappresentazione tra le due fasi sono parametrizzati direttamente tramite *P*, Ad esempio Map:

```
1 case Map[P <: Phase, A, B](upstream: StreamTree[P, A], ...) extends StreamTree[P, B]
```

Si noti come la fase del nodo impone anche la fase del relativo *upstream* garantendo la correttezza dell'intera struttura dati. Gli operatori la cui struttura viene invece effettivamente estesa hanno varianti specifiche per fase.

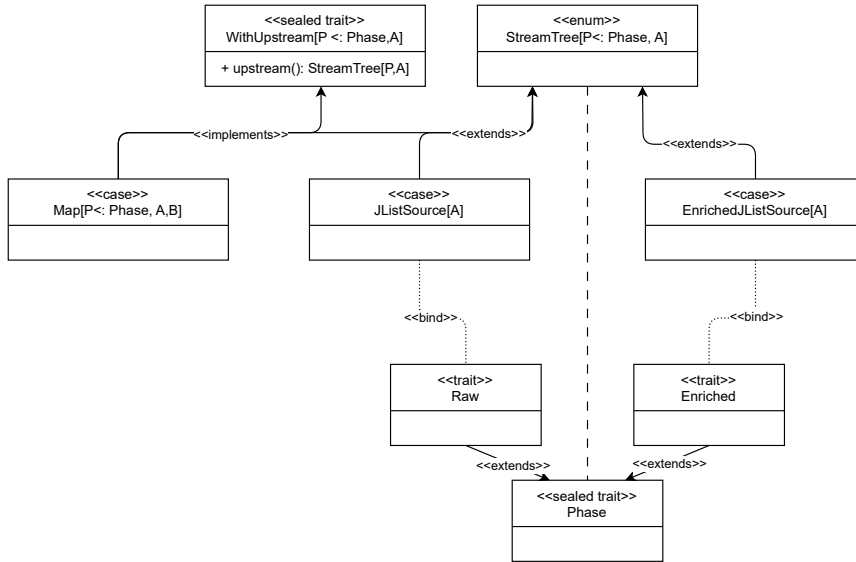


Figura 5.3: Struttura semplificata della Stream IR. La relazione non standard «bind», indica la specializzazione di un parametro di tipo mediante un tipo concreto. nel diagramma, il parametro di fase P viene specializzato con Raw o Enriched.

5.2.2 Ast e AstExt

StreamTree descrive la pipeline, ma le fasi del compilatore devono scambiarsi altre informazioni come strategia terminale e le informazioni statiche della pipeline. **Ast** e **AstExt** raccolgono rispettivamente il risultato del parsing e quello dell'enrichment.

Ast: il risultato del parsing

Ast costituisce il risultato della fase di parsing, è composto da:

parsedStream L'albero della pipeline in fase Raw.

collectionStrategy La rappresentazione dell'operazione terminale, descritta nella Sezione 5.4.

prefixStatements Le istruzioni Scala estratte dai blocchi che racchiudono l'espressione della pipeline.

executionMode La modalità **Sequential** o **Parallel**, determinata dal punto di ingresso della macro.

AstExt: il risultato dell'enrichment

AstExt raccoglie le informazioni utilizzate dal generatore della OP IR. E' composto da:

enrichedStream L'albero in fase **Enriched**, i cui nodi includono i riferimenti introdotti durante l'enrichment.

declarations Le dichiarazioni generate per materializzare espressioni e inizializzare lo stato della pipeline.

collectionStrategy La strategia terminale arricchita: **EnrichedCollectionStrategy** conserva la strategia originaria, l'eventuale riferimento booleano **earlyExitVar** per l'arresto anticipato del collector e la lista **ref** dei predicati che controllano la prosecuzione dell'elaborazione.

hasAlignedIndexes Indica se la posizione di un elemento nella sorgente può essere usata direttamente come posizione nel risultato.

cardinality Descrive il numero di elementi prodotti come dimensione esatta, limite superiore o quantità sconosciuta. Quando disponibile, la dimensione è rappresentata da un `Expr[Int]`.

5.3 Rappresentazione per la Code Generation (OP IR)

La Stream IR descritta nella sezione precedente mantiene una rappresentazione di alto livello della pipeline e dei suoi operatori. Tale struttura è particolarmente adatta alle analisi specifiche del dominio degli stream, ma non costituisce una rappresentazione conveniente per la generazione del codice.

Per questo si introduce una seconda rappresentazione intermedia, *Operation IR* (OP IR). La traduzione verso questa rappresentazione costituisce un primo processo di *lowering*: operatori di alto livello vengono eliminati e sostituiti da costrutti che descrivono esplicitamente il modello di esecuzione della pipeline. A differenza della Stream IR, la OP IR non modella quindi il dominio degli stream.

La rappresentazione è organizzata attorno a due categorie fondamentali: *Values*, che descrivono espressioni che producono un valore, e *Operations*, che descrivono azioni e strutture di controllo. Un programma nella OP IR è una sequenza di operazioni e un valore che ne costituisce il risultato finale.

Questo secondo linguaggio intermedio permette di suddividere la generazione del codice in due fasi con responsabilità distinte. La prima opera sulla rappresentazione logica della pipeline e determina la strategia di esecuzione più adatta esprimendola attraverso la OP IR. La seconda fase è invece meccanica: opera sulle primitive della OP IR e ne effettua il lowering nei corrispondenti costrutti Scala. Le decisioni relative a come implementare la pipeline sono quindi prese prima della generazione effettiva del codice.

5.3.1 Valori

I *Values* rappresentano espressioni che producono un risultato e sono utilizzati come operandi dalle operazioni della OP IR. La gerarchia è composta da valori elementari, come costanti o riferimenti a simboli e da valori composti che descrivono operazioni su altri valori come confronti, operazioni aritmetiche o applicazioni di funzioni.

Un **Value** descrive quindi un'espressione che può essere incorporata all'interno di una **Op**. Ad esempio, la condizione di un ciclo **While** è rappresentata come un **Value[Boolean]**, mentre il valore assegnato da una **AssignVal** è a sua volta rappresentato tramite un **Value**. Un esempio parziale della gerarchia è riportato in Figura 5.4.

Ogni valore possiede l'informazione relativa al tipo che rappresenta, permettendo quindi di esprimere staticamente i vincoli a livello di tipi tra le diverse primitive della IR. Per esempio in **LessThan**, entrambi gli operandi devono essere valori interi, mentre il risultato è necessariamente un **Value[Boolean]**.

La sola informazione contenuta nel parametro generico non è però sufficiente per la successiva generazione del codice: ogni **Value[T]** espone quindi anche un **Type[T]**, attraverso il campo **valueType**.

Si realizza così una forma di *type capture*: oltre a conoscere staticamente che un nodo produce un valore di tipo **T**, la IR conserva una rappresentazione del tipo utilizzabile dalle Scala Quotes. Per tipi noti in maniera statica o per valori parametrici, tale informazione viene propagata attraverso un parametro contestuale **Type[T]**.

Come approfondito nella Sezione 6.4.1, questa informazione viene poi sfruttata durante il lowering per costruire espressioni Scala correttamente tipizzate.

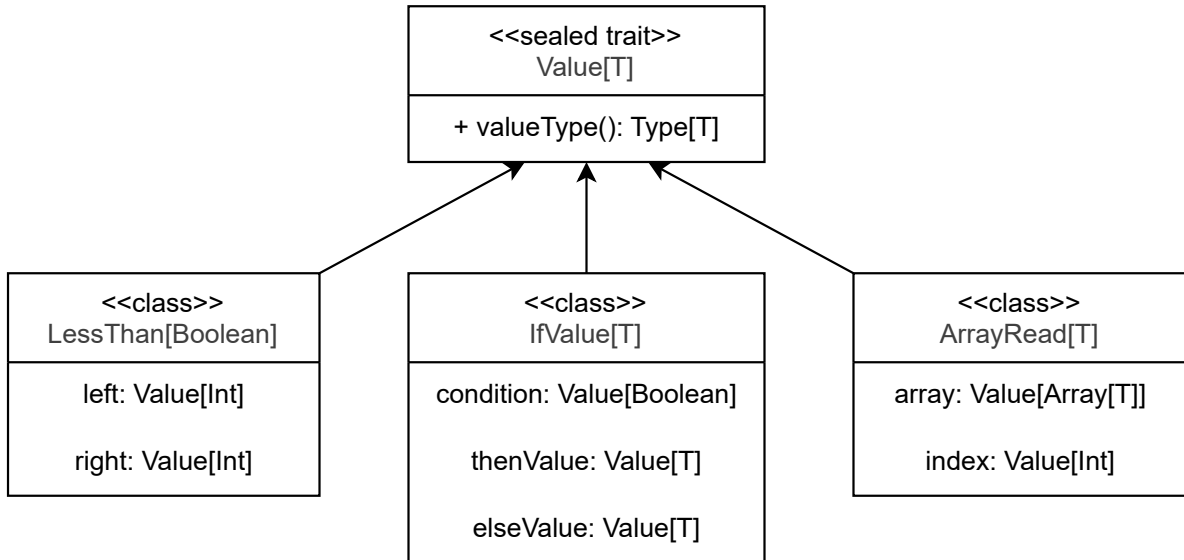


Figura 5.4: Gerarchia e rappresentazione dei valori nella OP IR.

5.3.2 Operazioni

Le *Operations* rappresentano invece le azioni che costituiscono il corpo del programma generato.

La OP IR comprende operazioni elementari per la dichiarazione e l'aggiornamento dello stato, come `Declare[T]`, `AssignVal[T]` o `Inc`. Il controllo del flusso è espresso esplicitamente attraverso `If` e `While`. Nel contesto parallelo, il nodo dedicato `Op.Parallel[OUT]` modella la partizione concorrente del carico e la riduzione finale tramite `ParallelCombine[OUT]`, svincolando la firma del nodo dal tipo interno del buffer di accumulo.

Sono inoltre presenti operazioni specializzate come ad esempio `ArrayWrite` che rappresenta la scrittura di un elemento in un array o `DynamicArrayBuilderAdd` che rappresenta l'inserimento in un buffer.

5.4 La collection strategy

La fase terminale di una stream è rappresentata internamente attraverso una `CollectionStrategy[A, R]` (Figura 5.5). Questa astrazione descrive la strategia con cui gli elementi prodotti dalla pipeline (di tipo `A`) devono essere consumati e aggregati per ottenere il risultato finale di tipo `R`. La firma del tratto è volutamente limitata a due parametri di tipo `[A, R]`, incapsulando il tipo intermedio del buffer `Buf` esclusivamente all'interno della variante generica `WithCollector[A, Buf, R]`.

I collector possono essere suddivisi in due categorie:

concrete collectors, che forniscono direttamente l'implementazione delle operazioni necessarie alla raccolta. Nella modalità di compilazione strict (`strictInlining = true`) sono richiesti un tipo concreto con nome visibile e metodi dichiarati `inline def`, così che Scala possa espandere le operazioni del collector direttamente nel ciclo di scansione. Disabilitando `strictInlining` sono ammesse anche normali invocazioni a runtime con classi anonime, la cui eventuale ottimizzazione è demandata al JIT. Il meccanismo di generazione è descritto nella Sezione 6.4.3.

intrinsic collectors. Questi non costituiscono l'implementazione effettivamente utilizzata a runtime, ma agiscono come placeholder all'interno della DSL. Il compilatore li riconosce durante il parsing e li sostituisce con una `CollectionStrategy` primitiva, la cui implementazione è definita direttamente all'interno della libreria. Appartengono a questa categoria le operazioni `toArray` e `summing`.

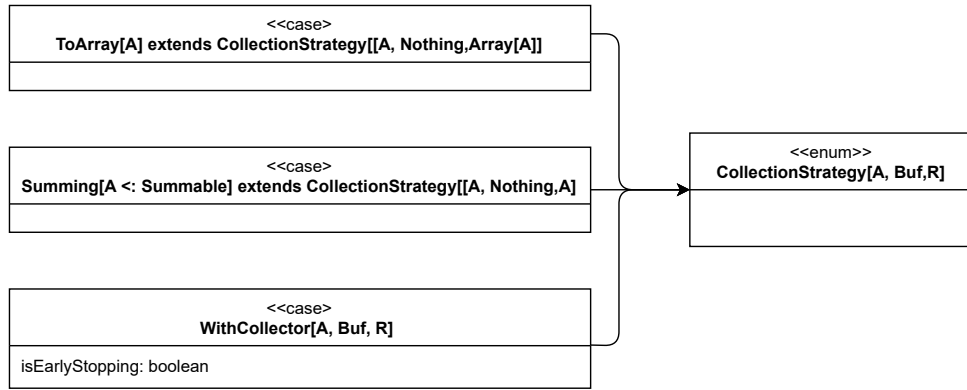


Figura 5.5: Diagramma UML delle classi per la gerarchia `CollectionStrategy[A, R]`. `Summable` è definito come union type dei tipi numerici `Int`, `Long`, `Float` e `Double`.

La strategia viene infine arricchita in `EnrichedCollectionStrategy[A, R]`, che combina la strategia selezionata con l'eventuale variabile booleana di arresto anticipato (`earlyExitVar`) e la lista di predicati di controllo (`ref`).

Capitolo 6

Implementazione

6.1 Parsing

La fase di parsing ha l'obiettivo di trasformare l'albero sintattico prodotto dal compilatore Scala in una `StreamIR` (Sezione 5.2) e di estrarre la `CollectionStrategy` dal `Collector`. Il parsing della pipeline e quello dell'operazione terminale sono mantenuti separati e vengono successivamente combinati nel contenitore `Ast` (Sezione 5.2.2), insieme alle istruzioni preliminari e alla modalità di esecuzione.

6.1.1 Parsing della stream

Il parser riconosce le operazioni all'interno dei nodi dell'AST Scala e ricostruisce ricorsivamente la struttura logica della pipeline. Un esempio completo della trasformazione dall'AST Scala alla corrispondente Stream IR è riportato nella Sezione 6.5.

Rappresentazione del risultato del parsing

Il risultato intermedio è rappresentato da `ParsedTree`, che consente al parser di restituire un albero senza che il chiamante debba conoscerne a priori il tipo di output. Quest'ultimo viene infatti determinato durante il parsing ed è rappresentato mediante l'abstract type member `Out`.

L'implementazione mantiene contemporaneamente `current: StreamTree[Phase.Raw, Out]` e `outType: Type[Out]`. Il primo conserva la relazione statica tra l'albero e il tipo degli elementi prodotti, mentre il secondo ne mantiene l'evidenza richiesta dalle API di `scala.quoted`.

Il tipo può essere recuperato nelle successive operazioni attraverso un path-dependent type:

```
1 type In = upstream.Out
2 given Type[In] = upstream.outType
```

Il parser può quindi tipare correttamente funzioni come `In => Boolean` per `filter` o `In => Out` per `map`.

Riconoscimento delle operazioni

Le operazioni della DSL vengono riconosciute mediante pattern matching sulla struttura dell'AST. In forma semplificata:

Listing 6.1: Riconoscimento delle operazioni della DSL

```
1 expr match
2   case Inlined(_, _, inner) => parseTerm(inner)
3   case Apply(TypeApply(NamedMethod("from"), _), List(source)) => ...
4   case Apply(Select(upstream, "filter"), List(predicate)) => ...
```

```

5 case Apply(Select(upstream, "skip"), List(n)) => ...
6 case Apply(Select(upstream, "limit"), List(n)) => ...
7 case Apply(TypeApply(Select(upstream, "map"), _, List(mapper)) => ...
8 case Apply(TypeApply(Select(upstream, "flatMap"), _, List(mapper)) => ...
9 case Apply(Select(upstream, "parallel"), _) => parseTerm(upstream)
10 case Block(statements, expr) => ...

```

Ogni caso analizza ricorsivamente l'upstream e costruisce il corrispondente nodo della Stream IR. Il tipo di output dell'upstream viene recuperato attraverso `upstream.Out` e la relativa evidenza `upstream.outType`, mantenendo tipizzate sia le funzioni ricevute dalla pipeline sia i nodi della IR.

Parsing ricorsivo di flatMap

`flatMap` richiede un trattamento specifico poiché la funzione ricevuta produce a sua volta una `Stream`, che deve essere sottoposta ricorsivamente allo stesso processo di parsing. Si consideri:

```

1 FusedStream
2   .from(numbers)
3   .flatMap(x =>
4     FusedStream.from(elements(x)).filter(y => y > 0)
5   )

```

Il parametro `x` rappresenta un elemento che sarà disponibile soltanto a runtime. Per poter analizzare staticamente la stream interna viene quindi creato un binder tipizzato che ne rappresenta simbolicamente il valore:

```

1 type In = upstream.Out
2 given Type[In] = upstream.outType
3
4 val flatMapBinder = createConstant[In]("flatMapElem")
5 val elemRef = Ref(flatMapBinder).asExprOf[In]

```

La funzione viene interpretata come `In => Stream[Out]` e applicata al binder mediante β -reduction:

```

1 val innerStreamExpr =
2   Expr.betaReduce('{ $function($elemRef) })
3
4 val innerTree = parseTerm(innerStreamExpr.asTerm)

```

Si ottengono così due sottoalberi: l'upstream esterno e la stream interna prodotta da `flatMap`. Il nodo della IR conserva inoltre il simbolo del binder che li collega. Durante la successiva code generation il binder verrà associato all'elemento corrente prodotto dall'upstream.

6.1.2 Parsing del collector

L'obiettivo di questa operazione è ricondursi a una collection strategy (approfondita nella Sezione 5.4) a partire dalla rappresentazione ricevuta. Il primo passo consiste nell'analizzarne il tipo associato all'espressione confrontandolo con i collector opachi della libreria, se è questo il caso il parser costruisce direttamente la strategia corrispondente. Nel caso generale, invece, il collector fornito dall'utente deve essere preservato, viene quindi estratta la *stop policy* per determinare staticamente se il collector supporti lo short-circuiting. Il risultato di questa analisi viene infine incorporato nella strategia generica `WithCollector`. La trasformazione può essere rappresentata come:

`ToArrayCollector[A] → ToArray[A]`

`SummingCollector[A] → Summing[A]`

`CollectorBase[A, Buf, R] → WithCollector[A, Buf, R]`

Nota la tipizzazione su `TerminationPolicy` a questo punto viene persa, è infatti utilizzata solo per definire la grammatica della dsl e viene convertita nel valore booleano `isEarlyStopping` dentro `WithCollector`

Prima di costruire la strategia generica, se `strictInlining` è abilitato, il parser esegue due controlli separati rifiutando classi anonime e verifica che le implementazioni di `supplier`, `accumulator` e `finisher` siano marcate con `inline`.

6.1.3 Parsing della configurazione statica

La libreria effettua un'ulteriore operazione di parsing relativa alla configurazione di compilazione. A differenza del parsing della stream, basato sull'analisi del TASTy, in questo caso l'estrazione viene effettuata mediante quoted pattern matching. Poiché la configurazione deve essere interamente nota a compile time, i valori dei suoi campi devono poter essere estratti staticamente dall'espressione. Sono quindi accettate istanziazioni di `CompileConfig` i cui argomenti siano costanti note in fase di compilazione, come mostrato dal seguente `FromExpr`:

Listing 6.2: Unapplier con quoted pattern matching per configurazione statica

```
1 given FromExpr[CompileConfig] with {
2   def unapply(expr: Expr[CompileConfig])(
3     using Quotes
4   ): Option[CompileConfig] = {
5     expr match {
6       // CompileConfig(unsafe, logging, strictInlining)
7       case '{ CompileConfig($u, $l, $i) } =>
8         for {
9           unsafe <- u.value
10          logging <- l.value
11          strictInlining <- i.value
12        } yield CompileConfig(unsafe, logging, strictInlining)
13
14      // new CompileConfig(unsafe, logging, strictInlining)
15      case '{ new CompileConfig($u, $l, $i) } =>
16        for {
17          unsafe <- u.value
18          logging <- l.value
19          strictInlining <- i.value
20        } yield CompileConfig(unsafe, logging, strictInlining)
21      case _ => None
22    }
23  }
24 }
```

6.2 Enrichment

Il contenitore `Ast` prodotto dal parser descrive la struttura logica della pipeline, ma non contiene ancora tutte le informazioni necessarie alla generazione del codice. La fase di *enrichment* completa quindi la `StreamTree` con informazioni determinate staticamente e restituisce un `AstExt` (Sezione 5.2.2), che raccoglie l'albero arricchito e le informazioni utilizzate nelle successive fasi di generazione.

L'enrichment svolge tre attività principali: genera le istruzioni di materializzazione delle espressioni che devono essere valutate una sola volta, introduce e propaga lo stato e le condizioni necessarie al controllo dell'esecuzione e determina proprietà globali della pipeline, quali la cardinalità del risultato e l'allineamento degli indici.

6.2.1 Materializzazione delle espressioni

Consiste nell'estrarre le espressioni la cui valutazione deve avvenire una sola volta prima dell'esecuzione della pipeline. Per esempio i parametri delle operazioni `skip` e `limit` quando presenti, vengono materializzati in variabili immutabili. In questo modo un'espressione come:

```
stream.limit(computeLimit())
```

non provoca la rivalutazione di `computeLimit()` a ogni iterazione.

Lo stesso principio viene applicato alle funzioni utilizzate da operatori come `map` e `filter`.

6.2.2 Stato e condizioni di terminazione

Durante l'enrichment vengono introdotte le informazioni di stato necessarie all'esecuzione degli operatori. Le operazioni `skip` e `limit` richiedono ad esempio un contatore che rappresenti la posizione corrente nel punto della pipeline in cui sono applicate. Nel caso di `limit`, lo stato ha anche effetto sulla terminazione della pipeline: raggiunto il numero massimo di elementi non è più necessario interrogare la sorgente. La condizione:

$$\text{sliceCounter} < \text{until}$$

viene quindi aggiunta all'insieme dei predicati di terminazione e propagata verso i nodi a monte.

Lo stesso meccanismo viene utilizzato per i collector *short-circuiting*. Quando la strategia di raccolta consente la terminazione anticipata, viene introdotta una variabile mutabile che rappresenta la possibilità di continuare l'elaborazione. Quando il collector determina che il risultato è completo, tale variabile viene modificata, causando l'arresto dei cicli interessati. La condizione di un ciclo generato non è quindi costituita solamente dalla condizione della sorgente, ma dalla combinazione con tutti i predicati propagati:

$$C_{\text{loop}} = C_{\text{source}} \wedge p_1 \wedge p_2 \wedge \dots \wedge p_n.$$

`FlatMap` rappresenta un caso particolare: il relativo nodo contiene sia l'upstream della pipeline esterna sia la stream interna prodotta dalla funzione. Quest'ultima viene arricchita a sua volta e le informazioni ottenute vengono associate al nodo.

L'enrichment determina pertanto il livello di annidamento al quale stato e condizioni dovranno essere collocati durante la successiva generazione del codice: lo stato appartenente alla stream interna deve essere infatti inizializzato nuovamente per ciascun elemento prodotto dalla stream esterna. Ad esempio, il contatore associato a un `limit` interno alla `flatMap` non può essere dichiarato globalmente, ma deve essere ricreato all'interno della corrispondente iterazione. Viceversa, una condizione di terminazione introdotta a valle della `flatMap` deve poter arrestare sia l'iterazione interna sia quella esterna.

6.2.3 Analisi della cardinalità

Terminato l'enrichment dell'albero vengono determinate due proprietà utilizzate per selezionare la strategia di generazione: la cardinalità del risultato e l'allineamento degli indici. La cardinalità è rappresentata mediante:

```

1 enum Cardinality {
2   case Exact(size: Expr[Int])
3   case UpperBound(size: Expr[Int])
4   case Unknown
5 }

```

Una sorgente indicizzabile, come un array o una lista, possiede cardinalità esatta. Operazioni come `map` la preservano, mentre `filter`, potendo eliminare elementi ma non aumentarne il numero, la trasformano in un limite superiore. Sorgenti genericamente iterabili e operazioni come `flatMap`, per le quali non è possibile determinare staticamente il numero di elementi prodotti, conducono invece a una cardinalità sconosciuta.

A questa informazione si affianca `hasAlignedIndexes`, ovvero se, in caso di sorgenti indicizzate, l'indice dell'elemento nella sorgente corrisponde all'indice nel risultato. La proprietà è inizialmente valida per sorgenti indicizzabili ed è preservata da operazioni come `map` e `limit`; viene invece invalidata da `filter`, `skip` e `flatMap`, che rompono la corrispondenza tra posizione letta e posizione prodotta.

6.3 Traduzione in OP IR

Una volta completato l'enrichment, la Stream IR contiene le informazioni necessarie per determinare come la pipeline debba essere generata. La fase successiva traduce quindi la rappresentazione logica della stream nella *Operation IR* (OP IR), introdotta nella Sezione 5.3. È in questa fase che vengono prese le principali decisioni relative al piano di esecuzione. La strategia generata dipende infatti dall'operazione terminale, dalla modalità sequenziale o parallela, dalla cardinalità della pipeline, dall'allineamento degli indici e dalle caratteristiche della sorgente. Il risultato è un `Program` composto da operazioni e da un valore finale, nel quale i concetti propri del dominio degli stream sono stati tradotti in dichiarazioni, condizioni, cicli e operazioni sui valori. La fase di lowering descritta nella Sezione 6.4 può quindi limitarsi alla traduzione meccanica di tali operazioni verso l'AST Scala.

6.3.1 Selezione della strategia terminale

Il primo livello decisionale riguarda l'operazione terminale. La `CollectionStrategy` ottenuta durante il parsing distingue tre casi:

- `ToArray`, per il quale viene selezionata una strategia di materializzazione dipendente dalla cardinalità della pipeline;
- `Summing`, tradotto in un accumulatore numerico specializzato sul tipo degli elementi;
- `WithCollector`, che utilizza il protocollo generale `supplier/accumulator/finisher`.

Nel caso del terminale `toArray`, per esempio, la strategia dipende principalmente dalla cardinalità e dall'allineamento degli indici. Con cardinalità esatta e indici allineati, è possibile preallocare l'array finale e utilizzare direttamente l'indice della sorgente. La stessa proprietà è particolarmente utile in parallelo: poiché la posizione finale di ogni elemento è nota indipendentemente, i worker possono scrivere in regioni distinte dello stesso array mediante `ParallelCombine.ArrayDirect()`, senza produrre risultati intermedi. Quando è noto solamente un limite superiore, ad esempio in presenza di `filter`, viene comunque preallocato un array, ma viene mantenuto un indice di output separato. Al termine l'array viene eventualmente troncato al numero di elementi effettivamente prodotti. Se invece la cardinalità è sconosciuta, come per una sorgente genericamente iterabile o per `flatMap`, viene utilizzato un buffer dinamico e l'array viene materializzato solamente al termine dell'elaborazione.

6.3.2 Generazione della pipeline e delle sorgenti

Determinata la strategia terminale, viene generato il corpo della pipeline. La generazione segue un modello basato su una funzione di emissione:

```
1 private type Emit[A] = Emitted[A] => Op
2
3 private final case class Emitted[A](
4   elem: Value[A],
5   sourceIndex: Option[Value[Int]]
6 )
```

La funzione `Emit` rappresenta il comportamento del downstream quando il nodo corrente produce un elemento. Gli operatori intermedi non generano quindi un ciclo autonomo, ma trasformano questa continuation e delegano la generazione al proprio upstream. Il ciclo viene emesso solamente quando la ricorsione raggiunge la sorgente.

Ad esempio, `filter` condiziona l'invocazione del downstream:

$$E'(x) = \text{if } p(x) \text{ then } E(x),$$

che nella OP IR corrisponde a:

```
1 If(
2   ApplyFun(filter.predicate, emitted.elem),
3   emit(emitted)
4 )
```

Una `map` modifica invece il valore propagato:

```
1 Declare(mapped, ApplyFun(map.function, emitted.elem))
2 emit(Emitted(SymbolRef(mapped), emitted.sourceIndex))
```

Il caso più articolato è `flatMap`. Il nodo contiene infatti una seconda Stream IR che rappresenta la pipeline interna. Il binder creato durante il parsing viene associato all'elemento corrente dell'upstream:

```
1 Declare(flatMap.elemSymbol, emitted.elem)
```

La stream interna viene quindi tradotta ricorsivamente utilizzando la stessa continuation del `flatMap`. Il risultato è l'annidamento diretto del ciclo interno nel ciclo esterno, senza materializzare una stream intermedia. Concettualmente:

```
1 while (outerCondition) {
2   val binder = nextOuter()
3   while (innerCondition) {
4     val y = nextInner(binder)
5     if (p(y))
6       emit(y)
7   }
8 }
```

La ricorsione termina infine quando viene raggiunta la sorgente, che introduce il ciclo effettivo. La tipologia concreta della sorgente può determinare ulteriori specializzazioni. Per una `java.util.List`, ad esempio, viene distinto il caso `ArrayList`, per il quale l'accesso indicizzato evita la creazione dell'iteratore e, se abilitato dalla configurazione di compilazione (`useUnsafe`), è inoltre possibile accedere direttamente all'array interno tramite `VarHandle`.

6.3.3 Generazione del piano parallelo

Nel caso parallelo, la OP IR raccoglie esplicitamente le informazioni necessarie a descrivere la partizione del lavoro:

- l'intervallo della sorgente interessato dalla computazione;
- la sequenza di operazioni da eseguire su ciascuna partizione;
- l'eventuale risultato locale prodotto da ogni partizione;
- la strategia di combinazione dei risultati parziali;
- il riferimento al risultato finale del calcolo.

La OP IR non descrive quindi i dettagli del meccanismo di scheduling: rappresenta il parallelismo come operazione di alto livello, lasciando alla successiva fase di lowering la sua traduzione nelle primitive runtime.

6.4 Lowering

La fase di lowering costituisce l'ultimo passaggio del compilatore: dato un `Program[T]`, produce un'espressione Scala `Expr[T]`, che viene restituita al compilatore scala. A questo punto le decisioni relative alla strategia di esecuzione sono già state prese: il lowering effettua una traduzione dell'OP IR nei corrispondenti nodi dell'AST Scala. La trasformazione è implementata principalmente attraverso due funzioni: `lowerOp`, per le operazioni, e `lowerValue`, per le espressioni che producono un valore.

6.4.1 Traduzione dell'OP IR

La traduzione viene effettuata mediante pattern matching esaustivo sui nodi dell'OP IR, le operazioni vengono ricondotte ai corrispondenti costrutti dell'AST Scala. Analogamente, i nodi `Value[T]` vengono tradotti ricorsivamente in `Expr[T]`. Riferimenti, costanti ed espressioni Scala già disponibili possono essere convertiti direttamente, mentre gli operatori composti richiedono il lowering ricorsivo dei propri operandi. Ad esempio:

```
1 LessThan(left, right)
```

viene tradotto in:

```
1 '{
2   ${lowerValue(left)} < ${lowerValue(right)}
3 }
```

Lo stesso procedimento viene applicato alle operazioni logiche, confronti, accessi ad array e alle operazioni sui collector, eliminando progressivamente l'OP IR fino a ottenere esclusivamente nodi appartenenti all'AST Scala.

Questa trasformazione conserva le informazioni di tipo: per costruire un `Expr[T]`, scala richiede che sia disponibile un'evidenza `Type[T]`. Per questo motivo, come descritto nella Sezione 5.3, ciascun `Value[T]` conserva, oltre al parametro di tipo, anche il `Type[T]` corrispondente nel campo `valueType`. Durante il pattern matching sull'OP IR tale informazione viene recuperata e resa disponibile localmente. Ad esempio, nel lowering di una scrittura su array:

```

1 case write: ArrayWrite[t] =>
2   given Type[t] = write.valueType
3
4   val array = lowerValue[Array[t]](write.array)
5   val index = lowerValue[Int](write.index)
6   val value = lowerValue[t](write.value)
7
8   '{ $array($index) = $value }

```

L'informazione conservata nella rappresentazione intermedia permette quindi di mantenere il collegamento tra il sistema di tipi dell'OP IR e quello delle Scala Quotes fino all'ultima fase della generazione.

6.4.2 Applicazione e inlining delle funzioni

Un caso particolarmente importante è costituito da **ApplyFun**. In questo caso viene utilizzata la β -reduction, descritta nella Sezione 3.3.3. Quando la funzione è una lambda, la β -reduction ne inserisce il corpo direttamente nel punto di utilizzo. Ad esempio:

```

1 ApplyFun(x => x * 2, elem)

```

viene ridotto a:

```

1 elem * 2

```

In questo modo la funzione non deve essere mantenuta come entità invocabile nel codice generato, eliminando il relativo dispatch.

6.4.3 Chiamate e inlining dei collector

Le operazioni sui collector vengono tradotte generando l'istruzione dell'AST che chiama il metodo associato sull'oggetto collector, ad esempio per l'accumulazione:

```

1 Select.overloaded(collector.asTerm, "accumulator", Nil, arguments).asExprOf[T]

```

Questo permette al compilatore Scala di fare inlining dei metodi (quando dichiarati inline def). A differenza della β -reduction utilizzata per **ApplyFun** in questo caso si lascia l'inlining al compilatore Scala. I factory methods dei collector predefiniti sono **transparent inline**, così da esporre il tipo delle istanze create.

6.4.4 Lowering del parallelismo

Il nodo **Parallel** costituisce il caso più articolato del lowering. La fase precedente ha già determinato la strategia di esecuzione e la modalità di combinazione dei risultati; il lowering deve quindi trasformare tale piano nel corrispondente codice concorrente. Il tipo di **ParallelCombine** determina l'implementazione specializzata da generare. La libreria prevede tre strategie specializzate e una generica:

- **Sum**: combina i risultati parziali mediante somma;
- **ArrayConcat**: concatena gli array prodotti dai singoli chunk;
- **ArrayDirect**: scrive direttamente nell'array finale condiviso;
- **GenericCombiner**: combina i risultati tramite il **Combinable** associato a un **Collector** generico.

L'intervallo della sorgente viene partizionato in chunk. Quando la strategia richiede risultati intermedi, questi vengono memorizzati separatamente per ciascun chunk e successivamente combinati. Nel caso della somma, ad esempio:

```
1 partials(chunkIdx) = localResult
```

Al termine dell'elaborazione i risultati parziali vengono ridotti sequenzialmente per ottenere il risultato finale.

Anche lo scheduling viene definito a runtime sulla base della dimensione della sorgente e della configurazione. Il numero di chunk è limitato dalla dimensione dell'input e determinato in funzione del numero di worker. L'assegnazione non è statica: ogni worker acquisisce un chunk e, terminata l'elaborazione, richiede dinamicamente il successivo attraverso un contatore atomico condiviso. In questo modo i chunk possono essere distribuiti progressivamente tra i worker senza fissarne preventivamente l'assegnazione e bilanciando dinamicamente il carico. Non è presente un meccanismo di work-stealing.

6.5 Esempio di traduzione end-to-end

Questa sezione ripercorre passo passo il processo di analisi e generazione del codice effettuato dalla libreria a partire da una semplice pipeline. L'intero processo segue il flusso dell'applicativo:

Scala AST $\longrightarrow$ StreamIR $\longrightarrow$ Enriched StreamIR $\longrightarrow$ Execution Plan $\longrightarrow$ OpIR $\longrightarrow$ Scala AST $\longrightarrow$ JVM byte

6.5.1 L'input alla libreria

La pipeline utilizzata come esempio è mostrata nel Listato 6.5.1:

```
1 FusedStream.from(numbers)
2   .map(f)
3   .filter(p)
4   .toArray
```

La macro riceve l'espressione sotto forma di AST Scala tipizzato; in particolare, il parser riceve l'albero mostrato nel Listato 6.5.1:

```
1 Inlined(
2   None,
3   Nil,
4   Apply(
5     Select(
6       Apply(
7         Apply(
8           TypeApply(
9             Select(
10              Apply(
11                TypeApply(
12                  Select(Ident("FusedStream"), "from"),
13                  List(Inferred())
14                ),
15                List(Ident("numbers"))
16              ),
17              "map"
18            ),
19            List(Inferred())
```

```

20     ),
21     List(
22         Block(
23             List(
24                 DefDef(
25                     "$anonfun",
26                     List(TermParamClause(List(ValDef("i", Inferred(), None)))),
27                     Inferred(),
28                     Some(
29                         Apply(
30                             Select(Ident("i"), "*"),
31                             List(Literal(IntConstant(2)))
32                         )
33                     )
34                 )
35             ),
36             Closure(Ident("$anonfun"), None)
37         )
38     ),
39     ),
40     List(
41         Apply(
42             TypeApply(Ident("given_NotMapToStream_T"), List(Inferred())),
43             List(Ident("value"))
44         )
45     ),
46     ),
47     "filter"
48 ),
49 List(
50     Block(
51         List(
52             DefDef(
53                 "$anonfun",
54                 List(TermParamClause(List(ValDef("i", Inferred(), None)))),
55                 Inferred(),
56                 Some(
57                     Apply(
58                         Select(
59                             Apply(
60                                 Select(Ident("i"), "%"),
61                                 List(Literal(IntConstant(2)))
62                             ),
63                             "=="
64                         ),
65                         List(Literal(IntConstant(0)))
66                     )
67                 )
68             )
69         ),
70         Closure(Ident("$anonfun"), None)
71     )
72 )
73 )
74 )

```

E l'espressione rappresentate il terminale

```
1 fuse.FusedStream.toArray[scala.Int]
```

6.5.2 Il risultato del parsing

La prima fase consiste nella traduzione dell'AST Scala nella rappresentazione intermedia ad alto livello descritta nella Sezione 5.2. La Stream IR corrispondente all'AST del Listato 6.5.1 è riportata nel Listato 6.3:

Listing 6.3: Rappresentazione ottenuta al termine del parsing

```
1 Filter(  
2   upstream = Map(  
3     upstream = JListSource(  
4       source = list,  
5       elementType = Int  
6     ),  
7     function = (i: Int) => i * 2,  
8     inputType = Int,  
9     outputType = Int  
10  ),  
11  predicate = (i: Int) => i % 2 == 0,  
12  elementType = Int  
13 )
```

Per questa pipeline vengono inoltre determinate la strategia terminale `ToArray[Int]` e la modalità `Sequential`. Non sono presenti `prefixStatements`.

6.5.3 L'output dell'enrichment

La `StreamIR` del Listato 6.3 viene quindi sottoposta alla fase di enrichment (Sezione 6.2). La `Enriched StreamIR` risultante è illustrata nel Listato 6.4:

Listing 6.4: `StreamIR` dopo la fase di enrichment

```
1 declarations = [  
2   // Dichiarazione della variabile dove viene materializzata la fonte  
3   Declare(source$macro$1, ScalaExpr(list)),  
4   // Dichiarazione della variabile che rappresenta la size della fonte  
5   Declare(sourceSize$macro$1, ScalaExpr(source$macro$1.size()))  
6 ]  
7  
8  
9 stream =  
10 Filter(  
11   upstream = Map(  
12     upstream = EnrichedJListSource(  
13       source = ScalaExpr(source$macro$1),  
14       size = ScalaExpr(sourceSize$macro$1),  
15       elementType = Int  
16     ),  
17     function = (i: Int) => i * 2,  
18     inputType = Int,  
19     outputType = Int  
20   ),  
21   predicate = (i: Int) => i % 2 == 0,  
22   elementType = Int  
23 )
```

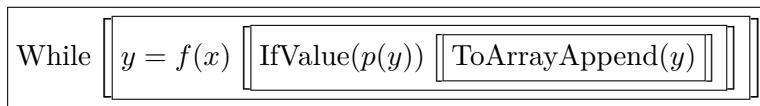
```

24 streamInfo =
25   StreamProperties(
26     // Abbiamo un filter quindi la size non e' esatta ma un upper bound
27     cardinality = UpperBound(ScalaExpr(sourceSize$macro$1)),
28     hasAlignedIndexes = false
29   )
30

```

6.5.4 Generazione dell'OpIR

A partire dalla rappresentazione arricchita del Listato 6.4, il compilatore determina la strategia con cui attraversare la sorgente ed eseguire le operazioni della pipeline. Il piano di esecuzione assume la forma:



Il piano di esecuzione viene espresso tramite OpIR nel Listato 6.5:

Listing 6.5: OPIR generato per una pipeline con mapping, filtro e raccolta in array

```

1 statements = [
2   // Estrae la lista di origine
3   Declare(source$macro$1, ScalaExpr(numbers)),
4   // Estrae la size della lista
5   Declare(sourceSize$macro$1, ScalaExpr(source$macro$1.size())),
6   // Crea un vettore preallocato
7   Declare(vec$macro$1, ArrayDefine[scala.Int](ScalaExpr(sourceSize$macro$1))),
8   // Dichiaro il counter e inizializza a 0
9   Declare(i$macro$1, ConstantVal(0)),
10  // Genera codice ottimizzato a runtime se e' una attaylist usiamo accesso
   // indicizzato (costo o(1), niente iteratori)
11  If(
12    condition = IsInstanceOf[java.util.ArrayList[]](ScalaExpr(source$macro$1)),
13    then =
14      CodeBlock(
15        Declare(i$macro$2, ConstantVal(0)),
16        While(
17          // Ciclo diretto su size, non abbiamo condizioni come limit o skip
18          condition = LessThan(SymbolRef(i$macro$2), ScalaExpr(sourceSize$macro$1)),
19          body =
20            CodeBlock(
21              // Leggiamo valore della list in una nuova variabile.
22              // Se useUnsafe = true allora JListRead legge direttamente
23              // l'array sottostante ad arraylist tramite varhandle
24              Declare(elem$macro$1,
25                JListRead(ScalaExpr(source$macro$1),
26                  SymbolRef(i$macro$2))),
27              // Map
28              CodeBlock(
29                Declare(
30                  mapped$macro$1,
31                  ApplyFun(((i: scala.Int) => i.*(2)), SymbolRef(elem$macro$1))
32                ),
33              // Filter
34              If(
35                condition = ApplyFun(

```

```

36         ((i: scala.Int) => i.%(2).==(0)),
37         SymbolRef(mapped$macro$1)),
38     then =
39         // Se la condizione e' rispettata write all'indice
40         CodeBlock(
41             ArrayWrite(
42                 SymbolRef(vec$macro$1),
43                 SymbolRef(i$macro$1),
44                 SymbolRef(mapped$macro$1)),
45             Inc(i$macro$1)
46         )
47     ),
48     ),
49     Inc(i$macro$2)
50 )
51 )
52 ),
53 //codice non ottimizzato, per List senza accesso diretto, usa iteratore
54 else =...
55 )
56 ],
57 result = IfValue(
58     // In base alla size dell'array lo ritorno direttamente o lo tronco
59     Equal(SymbolRef(i$macro$1), ArrayLength(SymbolRef(vec$macro$1))),
60     SymbolRef(vec$macro$1),
61     ArrayTake(SymbolRef(vec$macro$1), SymbolRef(i$macro$1))
62 )

```

6.5.5 Lowering verso l'AST Scala

L'elenco di operazioni viene poi tradotto nell'AST scala (qui omesso vista la verbosità e complessità). Segue invece una decompilazione del bytecode java corrispondente al codice generato:

```

1  ArrayList<Integer> numbers = new ArrayList<Integer>();
2  numbers.addAll(List.of(..));
3
4  int[] nArray;
5  RuntimeConfig conf = RuntimeConfig$.MODULE$.default();
6  ArrayList<Integer> source$macro$1 = numbers;
7  int sourceSize$macro$1 = source$macro$1.size();
8  int[] vec$macro$1 = new int[sourceSize$macro$1];
9  int i$macro$1 = 0;
10 if (source$macro$1 instanceof ArrayList) {
11     ArrayList<Integer> list$proxy1 = source$macro$1;
12     Object[] raw$macro$1 = ArrayListAccessor$.MODULE$.inline$ElementDataHandle().
13     get(list$proxy1);
14     for (int i$macro$2 = 0; i$macro$2 < sourceSize$macro$1; ++i$macro$2) {
15         int elem$macro$1 = BoxesRunTime.unboxToInt(raw$macro$1[i$macro$2]);
16         int mapped$macro$1 = elem$macro$1 * 2;
17         if (mapped$macro$1 % 2 != 0) continue;
18         vec$macro$1[i$macro$1] = mapped$macro$1;
19         ++i$macro$1;
20     }
21 } else {
22     // tramite foreach, rimosso per brevità
23 }
24 if (i$macro$1 == vec$macro$1.length) {

```

```

24     nArray = vec$macro$1;
25 } else {
26     Object object = Predef$.MODULE$.genericArrayOps(vec$macro$1);
27     nArray = (int[])ArrayOps$.MODULE$.take$extension(object, i$macro$1);
28 }
29 int[] result = nArray;

```

6.5.6 Il bytecode JVM generato

La corretta fusione delle operazioni può essere verificata anche ispezionando direttamente il bytecode JVM generato (riportato nel Listato 6.6):

Listing 6.6: Estratto del bytecode JVM relativo al corpo della pipeline

```

1 125: iload      10
2 127: iload      5
3 129: if_icmpge   175
4
5 132: aload      8
6 134: iload      10
7 136: aaload
8 137: invokestatic BoxesRunTime.unboxToInt
9 140: istore      11
10
11 142: iload      11
12 144: iconst_2
13 145: imul
14 146: istore      12
15
16 148: iload      12
17 150: iconst_2
18 151: irem
19 152: iconst_0
20 153: if_icmpne   169
21
22 156: aload      6
23 158: iload      7
24 160: iload      12
25 162: iastore
26 163: iinc        7, 1
27
28 169: iinc        10, 1
29 172: goto       125

```

Il risultato mostra in maniera evidente l'effetto della fusione e dell'inlining. L'operazione di mapping

$$x \mapsto x \cdot 2$$

non corrisponde più all'invocazione di una funzione, ma viene tradotta direttamente nell'istruzione JVM `imul`. Analogamente, il predicato

$$x \bmod 2 = 0$$

è rappresentato dalle istruzioni `irem` e `if_icmpne`.

Nel corpo del ciclo non sono presenti invocazioni dinamiche verso le funzioni utilizzate da `map` e `filter`. Non compaiono né chiamate a `Function.apply`, né istruzioni `invokedynamic`

associate alla costruzione o all'invocazione di lambda. Anche la raccolta del risultato è ridotta a operazioni primitive: **iastore** scrive direttamente il valore trasformato nell'array di output e **iinc** aggiorna il relativo indice.

Capitolo 7

Testing e valutazione delle prestazioni

7.1 Testing

Il testing deve affrontare due problemi:

1. **Correttezza a runtime:** accertarsi che il codice generato dalle macro produca i risultati attesi.
2. **Verifica a compile-time:** assicurarsi che le combinazioni non valide vengano intercettate subito dal compilatore, fornendo messaggi di errore chiari.

I test sono scritti con `MUnit` e sono organizzati su più livelli separando i controlli interni da quelli sull'API pubblica:

- **Unit test interni:** verificano i singoli stadi del motore di compilazione in isolamento.
- **Test funzionali end-to-end:** compilano ed eseguono pipeline complete scritte tramite l'API pubblica, verificando il risultato dell'esecuzione. Questi test sono suddivisi in quattro suite dedicate:
 - **SequentialStreamTest:** trasformazioni sequenziali di base e operatori di slice;
 - **PipelineSemanticsTest:** ordine di valutazione delle espressioni, isolamento dei side-effect e correttezza della fusione;
 - **SourceAndCollectorTest:** sorgenti Scala e Java, ottimizzazioni con accesso diretto alla memoria e collector terminali;
 - **ParallelStreamTest:** partizionamento del carico su più worker, concorrenza e riduzioni parallele.
- **Test di compilazione:** verificano che l'uso scorretto dell'API venga bloccato a tempo di compilazione.

Copertura

La copertura del codice è stata misurata tramite `sbt-scoverage` sul modulo principale (`core`), come riassunto in Tabella 7.1. La suite raggiunge una copertura complessiva delle istruzioni del **91.31%** (1733 su 1898 istruzioni) e una copertura dei rami decisionali del **91.67%**.

Tabella 7.1: Copertura del codice rilevata da `sbt-scoverage` sul modulo `core`.

Package / Sottosistema	Istruzioni	Coverage
<code>fuse.internal.parallel</code>	282 / 282	100.00%
<code>fuse.internal</code>	1336 / 1454	91.88%
<code>fuse.internal.ir</code>	86 / 94	91.49%
<code>fuse (API pubblica)</code>	29 / 68	42.65%
Totale (Branch: 91.67%)	1733 / 1898	91.31%

7.2 Valutazione delle prestazioni

7.2.1 Metodologia

Le misurazioni sono state condotte con (JMH) [6], su OpenJDK 64-Bit (21.0.7-LTS) x86_64 (Windows). I benchmark sono stati eseguiti con:

- Heap fissato a 8 GB sia come dimensione iniziale che massima (`-Xms8g -Xmx8g`), eliminando pause GC di espansione heap.
- Numero di thread/core attivi è stato vincolato a 4 (`-XX:ActiveProcessorCount=4`).
- 3 fork indipendenti della JVM, ciascuno preceduto da 5 iterazioni di warm-up da 2 secondi e seguito da 5 iterazioni di misura effettiva da 2 secondi (15 campioni per ciascun punto sperimentale).
- Salvo diversamente specificato, si opera con la configurazione di default (`useUnsafe = true`). Nella Sezione 7.2.4 viene discussa l'incidenza di questo flag.

Il comportamento della libreria viene comparato con tre implementazioni alternative:

1. **Java Stream:** le stream della stdlib di Java (`java.util.stream`).
2. **Scala View:** le collezioni lazy della stdlib di Scala 3 (`scala.collection.View`).
3. **Loop Java manuale:** codice imperativo Java classico scritto a mano con ciclo `while` e accumulatore locale.

7.2.2 Benchmark sequenziali

Per comprendere le caratteristiche della libreria è fondamentale distinguere tra un microbenchmark sintetico e uno più simile carico applicativo reale.

Scenario Monomorfico

Molti benchmark che hanno come oggetto le stream Java si limitano a valutare una singola, semplice pipeline (ad esempio `map` seguita da `sum`):

$$\text{Source} \xrightarrow{\text{map}(t \rightarrow t.\text{amount})} \xrightarrow{\text{sum}()} \text{Double}$$

In questo scenario artificiale, il compilatore JIT C2 osserva un unico target (i call site sono strettamente *monomorfici*) e riesce quindi ad applicare inlining speculativo e devirtualizzazione sia alle lambda sia all'infrastruttura a oggetti di Java Stream.

Tuttavia, come emerge chiaramente dai dati in Tabella 7.2, persino in questo contesto ideale Java Stream evidenzia limiti:

- Su collezioni piccole l’inizializzazione della pipeline introduce un costo fisso che rende Java Stream da 6.7 a oltre 11 volte più lenta di **FusedStream** e del loop manuale.
- Su volumi consistenti ($N = 100\,000$), nonostante l’inlining di C2, Java Stream non raggiunge la velocità del codice imperativo classico.

Tabella 7.2: Benchmark Monomorfo: tempo medio di esecuzione ($\mu\text{s/op}$).

Implementazione	$N = 10$	$N = 100$	$N = 1\,000$	$N = 100\,000$
FusedStream	0.006 $\pm$ 0.001	0.067 $\pm$ 0.001	0.914 $\pm$ 0.002	504.66 $\pm$ 11.23
Java Manual Loop	0.007 $\pm$ 0.001	0.068 $\pm$ 0.001	0.918 $\pm$ 0.002	507.56 $\pm$ 6.60
Scala View	0.042 $\pm$ 0.001	0.331 $\pm$ 0.006	3.230 $\pm$ 0.021	335.44 $\pm$ 3.46
Java Stream	0.068 $\pm$ 0.001	0.451 $\pm$ 0.005	4.290 $\pm$ 0.007	857.71 $\pm$ 16.89

Scenario Megamorfo (Carico applicativo reale)

Questo scenario simula un carico di analisi e reportistica su dati in-memory, eseguendo 8 query eterogenee su dati rappresentanti transazioni finanziarie. Ciascuna delle 8 query è isolata in un metodo statico dedicato (`@static def`) invocato in sequenza dal metodo di benchmark, garantendo che ogni pipeline venga compilata e ottimizzata in modo modulare dal compilatore JIT.

Nelle applicazioni reali, più pipeline distinte invocano operatori con lambda differenti. In Java Stream e Scala View i call site interni delle interfacce funzionali diventano *megamorfici* (più di due classi concrete osservate a runtime). La inline cache della JVM collassa, perdendo inlining speculativo e escape analysis. La JVM è costretta a eseguire dispatch virtuali ad ogni singolo elemento. Come riportato in Tabella 7.3, le prestazioni di **FusedStream** equivalgono a quelle del codice Java manuale lungo tutti gli ordini di grandezza analizzati, dimostrando la completa rimozione dell’overhead funzionale tramite inlining a compile-time.

Tabella 7.3: Benchmark Megamorfo Sequenziale: 8 query eterogenee ($\mu\text{s/op}$).

Implementazione	$N = 10$	$N = 100$	$N = 1\,000$	$N = 100\,000$
FusedStream (Unsafe)	0.012 $\pm$ 0.001	0.087 $\pm$ 0.003	1.489 $\pm$ 0.146	131.53 $\pm$ 13.73
FusedStream (Safe)	0.012 $\pm$ 0.001	0.085 $\pm$ 0.003	1.278 $\pm$ 0.009	126.98 $\pm$ 16.58
Java Manual Loop	0.011 $\pm$ 0.001	0.083 $\pm$ 0.001	1.270 $\pm$ 0.005	119.20 $\pm$ 7.06
Java Stream	0.089 $\pm$ 0.001	0.504 $\pm$ 0.019	4.649 $\pm$ 0.134	466.33 $\pm$ 2.51
Scala View	0.061 $\pm$ 0.001	0.485 $\pm$ 0.006	4.805 $\pm$ 0.121	639.96 $\pm$ 20.52

Analisi dei risultati. I dati sperimentali riportati in Tabella 7.3 evidenziano le prestazioni delle diverse soluzioni a confronto:

1. **Dominio prestazionale su tutte le scale:** sia **FusedStream** sia il ciclo imperativo Java manuale mostrano tempi di esecuzione equivalenti lungo tutti i valori di N . Su collezioni di dimensioni ridotte ($N \leq 100$), **FusedStream** supera Java Stream fino a **7.4** $\times$ e Scala View fino a **5.1** $\times$, grazie alla completa rimozione a compile-time delle allocazioni di descrittori di stream e iteratori intermedi.
2. **Scalabilità sui grandi volumi:** su collezioni di 100 000 elementi, **FusedStream** risulta **3.5** $\times$ più rapido di Java Stream e **4.9** $\times$ più rapido di Scala View.

3. **Parità tra accesso sicuro e Unsafe:** il confronto diretto tra `FusedStream (Unsafe)` e `FusedStream (Safe)` dimostra che con C2 attivo, il costo di accesso agli elementi della lista viene ottimizzato dal compilatore JIT, rendendo la versione fase analoga.

7.2.3 Benchmark paralleli

La valutazione del runtime parallelo analizza l'efficienza nell'esecuzione su un pool di 4 core fisici. Vengono messi a confronto:

- `FusedStream.parallel()`: partizionamento statico indicizzato con chunk distribuiti dinamicamente
- `Java parallelStream()`: parallelismo nativo di Java basato su decomposizione ricorsiva con `Splitterator` su `ForkJoinPool`.
- `Java Manual Parallel`: chunking manuale dell'array con sottomissione concorrente di task Callable tramite `invokeAll` su `ForkJoinPool`.

I risultati sono riassunti in Tabella 7.4.

Tabella 7.4: Benchmark Megamorfo Parallelo: 8 query eterogenee ($\mu\text{s}/\text{op}$).

Implementazione	$N = 10$	$N = 100$	$N = 1\,000$	$N = 100\,000$
FusedStream Parallel	10.66 ± 0.32	13.11 ± 0.61	14.32 ± 0.57	435.77 ± 12.67
Java Manual Parallel	11.27 ± 0.50	12.72 ± 0.59	13.16 ± 0.67	617.78 ± 13.27
Java Parallel Stream	3.75 ± 0.14	4.90 ± 0.21	8.83 ± 0.48	1029.31 ± 16.26

Speedup, efficienza e analisi di scalabilità a 4 core. Dall'analisi comparativa tra il benchmark sequenziale di riferimento a Tier 4 e il benchmark parallelo (Tabella 7.4) sulla taglia massima ($N = 100\,000$), emergono i seguenti indici di scalabilità:

- **FusedStream:** tempo sequenziale di riferimento $799.50\,\mu\text{s}$, tempo parallelo $435.77\,\mu\text{s}$. Lo speedup è pari a **1.83** $\times$, corrispondente a un'efficienza parallela del **45.8%** (rispetto all'accelerazione ideale di $4\times$ sui 4 core).
- **Java Manual Parallel:** tempo sequenziale di riferimento $758.64\,\mu\text{s}$, tempo parallelo $617.78\,\mu\text{s}$. Speedup pari a **1.23** $\times$, con un'efficienza del **30.7%**.
- **Java Parallel Stream:** tempo sequenziale di riferimento $1261.43\,\mu\text{s}$, tempo parallelo $1029.31\,\mu\text{s}$. Speedup pari a **1.23** $\times$, con un'efficienza del **30.6%**.

I risultati offrono chiarimenti cruciali sulle dinamiche della concorrenza su JVM:

1. **Perché Java Parallel Stream scala poco ($1.23\times$):** Java Parallel Stream divide ricorsivamente il flusso tramite `Splitterator`, allocando oggetti `ForkJoinTask` e introducendo continuo overhead di work-stealing. Poiché l'elaborazione di $100\,000$ elementi richiede solo circa $1\,\text{ms}$, il costo di allocazione dei task e coordinamento tra i thread rivalessa con il tempo effettivo di calcolo.
2. **Perché FusedStream non supera il 46% di efficienza:** benché `FusedStream.parallel()` superi Java Parallel Stream di ben $2.36\times$ grazie al chunking statico lineare senza allocazioni nel loop, l'efficienza teorica del 100% non è raggiungibile per due fattori determinanti:

- *Memory Bandwidth Wall*: l'iterazione parallela di 4 thread su 100 000 riferimenti ad oggetti e la scansione dei campi in memoria satura la banda passante del bus verso la RAM e genera traffico di coerenza nella cache L3 condivisa. Il calcolo (filtro su interi e somma di double) è estremamente leggero rispetto al costo di recupero dei dati dalla memoria principale, ricadendo nei limiti imposti dalla Legge di Amdahl per carichi memory-bound.
 - *Overhead di dispatch e riduzione finale*: la sottomissione dei chunk all'`ExecutionContext`, l'attesa su `CountDownLatch` e la successiva riduzione sequenziale ordinata dei risultati parziali rappresentano una frazione seriale non azzerabile.
3. **Comportamento su taglie ridotte** ($N \leq 1000$): per taglie contenute la parallelizzazione è sistematicamente controproducente ($10\text{--}14\ \mu\text{s}$ contro $0.01\text{--}1.4\ \mu\text{s}$ del sequenziale), dimostrando come l'attivazione del multi-threading debba essere riservata a dataset estesi. Su tali taglie minime, Java Parallel Stream risulta marginalmente più rapido per via dell'integrazione diretta con primitive di basso livello di `ForkJoinPool`, che riducono i context switch rispetto a un `ExecutionContext` generico sincronizzato via `CountDownLatch`.

7.2.4 Comportamento a freddo e livelli di compilazione JIT

Nelle applicazioni moderne (funzioni serverless o microservizi), le prestazioni durante l'avvio a freddo e i primi cicli di esecuzione sono rilevanti per contenere la latenza iniziale (*cold start*).

L'architettura di compilazione HotSpot (*Tiered Compilation*) prevede cinque livelli di esecuzione, controllabili tramite il flag JVM `-XX:TieredStopAtLevel=$tier`:

- **Tier 0**: interpretazione pura del bytecode JVM istruzione per istruzione.
- **Tier 1**: compilazione nativa immediata tramite il compilatore Client C1 senza profilazione (`-XX:TieredStopAtLevel=1`).
- **Tier 2 e Tier 3**: compilazione C1 con profilazione di base e completa (contatori di invocazione e branch).
- **Tier 4**: compilazione Server C2 con inlining speculativo, escape analysis e vettorizzazione SIMD (`-XX:TieredStopAtLevel=4`).

I livelli Tier 2 e Tier 3 rappresentano fasi di transizione in cui la JVM raccoglie profili prima della promozione a C2. Per questa ragione, la Tabella 7.5 confronta i tre livelli fondamentali: l'interprete puro (Tier 0), il compilatore C1 (Tier 1) e il compilatore C2 (Tier 4), tutti vincolati a 4 core 8 GB di heap.

I risultati sperimentali mettono in luce considerazioni teoriche e pratiche fondamentali:

1. **Efficienza a Tier 0 e parità di accesso a `ArrayList`**: in modalità interpretata pura (Tier 0), `FusedStream` risulta oltre 2 volte più veloce perfino del ciclo Java manuale ($52.12\ \mu\text{s}$ contro $110.64\ \mu\text{s}$ su $N = 1000$, e $5.57\ \text{ms}$ contro $11.54\ \text{ms}$ su $N = 100\,000$). L'origine di questo divario risiede nella tecnica di accesso a `ArrayList`: con la configurazione di default `useUnsafe = true` (che sfrutta il flag `-add-opens`), `FusedStream` accede direttamente al vettore `elementData` tramite istruzioni bytecode `aaload`, azzerando i controlli di bounds. Nel loop Java manuale, invece, ogni elemento viene recuperato invocando `list.get(i)` con un'istruzione `invokevirtual`, che in modalità interpretata comporta un pesante overhead di call-frame e controllo dei limiti. Disabilitando l'accesso non sicuro con `useUnsafe = false`, `FusedStream` emette a sua volta chiamate `list.get(i)`, allineandosi perfettamente alle prestazioni del ciclo manuale.

Tabella 7.5: Confronto prestazioni sui livelli JIT nello scenario Megamorfico ($\mu\text{s/op}$).

Configurazione JIT	FusedStream	Java Manual	Java Stream	Scala View
<i>Taglia $N = 10$:</i>				
Tier 0 (Interprete puro)	0.901 ± 0.013	1.240 ± 0.049	4.839 ± 0.057	14.519 ± 0.806
Tier 1 (C1)	0.041 ± 0.001	0.037 ± 0.001	0.331 ± 0.003	0.544 ± 0.005
Tier 4 (C2 completo)	0.012 ± 0.001	0.011 ± 0.001	0.089 ± 0.001	0.061 ± 0.001
<i>Taglia $N = 100$:</i>				
Tier 0 (Interprete puro)	5.561 ± 0.096	11.179 ± 0.653	16.342 ± 0.599	47.982 ± 0.952
Tier 1 (C1)	0.371 ± 0.005	0.383 ± 0.001	1.159 ± 0.005	2.223 ± 0.062
Tier 4 (C2)	0.087 ± 0.003	0.083 ± 0.001	0.504 ± 0.019	0.485 ± 0.006
<i>Taglia $N = 1000$:</i>				
Tier 0 (Interprete puro)	52.12 ± 0.39	110.64 ± 3.93	125.72 ± 2.81	374.04 ± 10.85
Tier 1 (C1)	3.68 ± 0.11	3.88 ± 0.02	8.71 ± 0.03	18.29 ± 0.07
Tier 4 (C2)	1.49 ± 0.15	1.27 ± 0.005	4.65 ± 0.134	4.81 ± 0.121
<i>Taglia $N = 100\,000$:</i>				
Tier 0 (Interprete puro)	5571.19 ± 191.09	11540.42 ± 226.61	12488.01 ± 415.40	37431.55 ± 700.42
Tier 1 (C1)	1172.27 ± 21.24	1239.64 ± 14.59	1575.74 ± 13.49	1851.88 ± 45.52
Tier 4 (C2)	131.53 ± 13.73	119.20 ± 7.06	466.33 ± 2.51	639.96 ± 20.52

- Prestazioni immediate a Tier 1:** non appena il compilatore C1 interviene (Tier 1), anche senza profilazione né analisi di inlining avanzata, **FusedStream** raggiunge immediatamente prestazioni equivalenti al ciclo Java manuale ($3.68 \mu\text{s}$ contro $3.88 \mu\text{s}$ su $N = 1\,000$). Al contrario, Java Stream e Scala View sono da $2.4\times$ a $5\times$ più lente, confermando la loro dipendenza critica dalle ottimizzazioni speculative di C2.
- Comportamento di warm-up e azzeramento del debito funzionale:** il passaggio da Tier 0 a Tier 4 registra un'accelerazione di diversi ordini di grandezza per tutte le implementazioni (da 5.57 ms a 0.13 ms per **FusedStream**), riflettendo il fisiologico guadagno derivante dalla traduzione in codice macchina nativo e dall'allocazione nei registri CPU. Tuttavia, ciò che **FusedStream** azzerava completamente è il *warm-up funzionale*: poiché il bytecode generato alla compilazione è già nella forma canonica di un ciclo imperativo privo di iteratori intermedi e closure, la JVM non deve attraversare cicli di de-ottimizzazione, né attendere che C2 tenti speculativamente di monomorfizzare call site megamorfici o di eliminare wrapper tramite Escape Analysis. Fin dal primo ciclo interpretato, **FusedStream** esibisce la stessa semantica esecutiva di un loop a basso livello.

7.2.5 Allocazioni e comportamento del Garbage Collector

Oltre al throughput, un fattore critico per le prestazioni stabili in produzione è l'impatto sul Garbage Collector (GC). Le librerie funzionali convenzionali producono un flusso continuo di oggetti a vita breve:

- Istanze di closure e lambda:** se le funzioni catturano variabili dall'ambiente circostante, la JVM è costretta ad allocare un'istanza sullo heap ad ogni valutazione della query.
- Descrittori di pipeline:** ogni stadio intermedio (`map`, `filter`) in Java Stream alloca un nodo `ReferencePipeline` e le relative classi interne.
- Boxing di primitivi:** in assenza di stream primitivi specializzati ogni valore transita attraverso oggetti wrapper.
- Wrapper di iteratori:** in Scala View ogni operatore crea un nuovo oggetto `Iterator` anonimo che incapsula l'iteratore a monte.

Questo induce frequenti cicli di garbage collection e fluttuazioni nella latenza di risposta.

FusedStream riduce le allocazioni introdotte dalla rappresentazione e dalla composizione della pipeline:

- Il ciclo di attraversamento è un costrutto nativo **while** senza iteratori intermedi;
- I valori trasformati vengono mantenuti in variabili locali, facilmente eliminate dal JIT compiler;
- I collector primitivi dedicati (come **summing**) accumulano direttamente su variabili primitive scalari (**double**, **int**), eliminando totalmente il boxing;

Per pipeline a tipi generici su collezioni a oggetti o per collector personalizzati non specializzati su tipi primitivi, le allocazioni di boxing e memorizzazione a runtime seguono la semantica standard del sistema dei tipi della JVM.

Capitolo 8

Retrospettiva

In questo capitolo vengono riassunte le considerazioni emerse durante lo sviluppo di `FusedStream`, analizzando i compromessi tecnici adottati, i limiti attuali della soluzione e le prospettive di evoluzione futura.

8.1 Limiti e compromessi progettuali

La progettazione di un motore di fusione a compile-time su JVM impone un bilanciamento continuo tra espressività dell'API, complessità delle macro e prestazioni del codice generato. Le principali scelte di compromesso hanno riguardato:

- **Costo di compilazione macro e dimensione del bytecode:** l'elaborazione dell'AST a compile-time e l'inlining aggressivo delle lambda tramite beta-reduction spostano l'onere computazionale sul compilatore Scala 3. Misurazioni sperimentali condotte tramite staging su JVM mostrano che la compilazione di una pipeline tipica sequenziale (`filter + map + summing`) richiede circa 128 ms, emettendo un loop bytecode estremamente compatto di appena 40–55 byte. Al contrario, pipeline con `flatMap` annidati richiedono circa 375 ms di tempo di compilazione ($2.9\times$ superiore) per via della scomposizione ricorsiva dell'AST e dell'elaborazione di due livelli di continuazioni, generando un bytecode di circa 110–130 byte. Sebbene il tempo di compilazione sia superiore a quello richiesto da librerie a runtime, esso viene ammortizzato istantaneamente all'avvio dell'applicazione.
- **Allocazione eager dell'upper bound in `toArray`:** quando una sorgente filtrata o generata da `flatMap` (la cui cardinalità esatta è incognita ma limitata superiormente da una dimensione massima nota) viene raccolta tramite `toArray`, la macro alloca preventivamente un array pari alla dimensione massima dell'intervallo. Questo compromesso privilegia la totale assenza di riallocazioni e ridimensionamenti dinamici all'interno del ciclo di attraversamento, a fronte di un potenziale consumo temporaneo di memoria qualora il predicato di filtro scarti un'elevata percentuale di elementi prima della compattazione finale.
- **Rigidità sintattica della pipeline (divieto di assegnazione a `val`):** poiché la macro necessita dell'intero albero sintattico per effettuare la fusione, una pipeline `FusedStream` non può essere assegnata a una variabile intermedia (`val`) né spezzata o composta dinamicamente attraverso funzioni separate. La catena da `FusedStream.from(...)` a `.collect(...)` deve costituire un'unica espressione continua al call-site. Per casi d'uso dinamici, la libreria fornisce il modulo disgiunto `stream-fusion-runtime`.
- **Operazioni non implementate rispetto a Java Stream:** l'API fused privilegia operazioni che possono essere trasformate direttamente in un unico ciclo di scansione lineare senza accumulo in memoria. Di conseguenza, operazioni che richiedono ordinamento, deduplicazione o memorizzazione intermedia globale (quali `sorted`, `distinct`, `count`, `reduce`,

`min/max`, `anyMatch`, `allMatch`, `takeWhile`, `dropWhile`, `peek`, `zip`) non sono attualmente supportate nella macro, così come la raccolta parallela nativa in collezioni dinamiche `toList/toSet` (che richiede un collector custom).

- **Non-associatività delle somme in virgola mobile parallele:** nella riduzione parallela di stream di `Double` o `Float` tramite `parallel().collect(summing)`, la partizione in chunk e l'ordine di riduzione dei contributi parziali dipendono dal numero di worker e da `chunksPerWorker`. A causa della non-associatività intrinseca dello standard IEEE 754, il valore numerico della somma può differire negli ultimi bit significativi rispetto alla valutazione strettamente sequenziale.
- **Contatori di slice e indici in `Int`:** a differenza delle Stream Java (in cui `skip` e `limit` operano su `long`), l'API e i contatori interni di `FusedStream` adottano esclusivamente il tipo primitivo `Int`. Questa scelta architetturale mantiene il bytecode compatto e massimizza l'efficienza sui registri della CPU nei contesti reali in-memory, validando a runtime la non negatività dei parametri (`IllegalArgumentException`). Di converso, su sorgenti lazy potenzialmente infinite che richiedano uno skip superiore a $2^{31} - 1$ elementi, il contatore interno incorrerebbe in overflow aritmetico.

8.2 Sviluppi futuri

Le evidenze raccolte durante la validazione sperimentale aprono interessanti direzioni di miglioramento:

- **Ottimizzazione dello scheduler parallelo:** come illustrato nella Sezione 7.2.3, Java Parallel Stream sfrutta `CountedCompleter` e ottiene maggiore efficienza sulle taglie intermedie. L'attuale architettura di `FusedStream` si appoggia a un `ExecutionContext` standard sincronizzato via `CountDownLatch`. Un'indagine su uno scheduler dedicato basato direttamente su primitive `ForkJoinPool` costituisce una priorità per abbattere l'overhead di sottomissione; tale approfondimento non è stato possibile per motivi di tempo.
- **Vettorizzazione SIMD:** con le nuove API del JDK (Project Panama e la *Vector API*), il generatore potrebbe emettere istruzioni vettoriali esplicite per le trasformazioni e le riduzioni aritmetiche su tipi primitivi, superando i limiti dell'auto-vettorizzazione di HotSpot C2.
- **Estensione dei collector paralleli:** introduzione di collector paralleli complessi con partizionamento hash (quali `groupBy` o aggregazioni associative multi-chiave), progettati per limitare le contese senza creare collezioni intermedie temporanee.

8.3 Conclusioni

Il lavoro svolto ha dimostrato la fattibilità e l'efficacia di un'architettura di *stream fusion* a tempo di compilazione in Scala 3. Sfruttando le macro tipizzate, la reflection a compile-time e l'inlining delle lambda, `FusedStream` riesce a colmare il divario storico tra la chiarezza dichiarativa della programmazione funzionale e la velocità dei loop imperativi scritti a mano su JVM.

La valutazione empirica ha confermato che, mentre le soluzioni esistenti degradano sensibilmente in scenari applicativi reali per via del fallimento dell'inline caching sui call site megamorfici, la fusione a compile-time mantiene prestazioni invarianti e competitive con il codice Java manuale a zero allocazioni.

Bibliografia

- [1] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. Stream fusion: From lists to streams to nothing at all. *ACM SIGPLAN Notices*, 42(9):315–326, 2007.
- [2] Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. Stream fusion, to completeness. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 285–299, 2017.
- [3] Dmitry Konstantinov. CASSANDRA-21289: Avoid capturing lambda allocation in UnfilteredSerializer.deserializeRowBody. Apache Cassandra Issue Tracker, 2026. Accessed: 2026-09-27.
- [4] Dmitry Konstantinov. CASSANDRA-21588: Stop using java streams in CIDRPermissions.SubsetPermissions#canAccessFrom hot method. Apache Cassandra Issue Tracker, 2026. Accessed: 2026-09-27.
- [5] Shayan Najd and Simon Peyton Jones. Trees that grow. *Journal of Universal Computer Science*, 23(1), 2017.
- [6] OpenJDK. Java Microbenchmark Harness (JMH). <https://github.com/openjdk/jmh>, 2024.
- [7] John Rose and Project Valhalla Team. The saga of the parametric vm. Technical report, OpenJDK Project Valhalla, 2021. June 2020–April 2021.
- [8] Nicolas Stucki, Jonathan Immanuel Brachthäuser, and Martin Odersky. Multi-stage programming with generative and analytical macros. In *Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, pages 110–122, 2021.
- [9] Nicolas Alexander Stucki. *Scalable metaprogramming in scala 3*. PhD thesis, EPFL, 2023.